L1 Intro: modalities

Medical imaging = many **modalities**, mostly non-invasive, for diagnosis and (surgery) planning • **X-ray**: cheap, global view; no fine organ detail; radiation • ★ **Ultrasound (US)**: sound waves; cheap, portable, no radiation or sedation; **operator dependent** (→ **generalises worst to new sites**); air and bone block the view • **CT** (Computed Tomography) = rotating X-ray → 3D; fast, good organ and bone detail, quantitative intensities (Hounsfield units); radiation, may need contrast • ★ **MRI** = magnet + radio waves; best soft-tissue contrast, any plane, no radiation; **slow → sedation for young children**, expensive • **PET** (Positron Emission Tomography) = injected radioactive tracer (e.g. radioactive glucose) emits gamma rays → shows **function, not anatomy**; blurry, so combined with CT/MRI; radiation • radiation: X-ray, CT, PET (contrast scans maybe); none: US, MRI, endoscopy, histopathology • invasive: endoscopy, histopathology (biopsy); minimally: PET, contrast agents • dose unit **Sievert (Sv)**: banana 0.1 μSv, arm X-ray 1 μSv, dental 5 μSv, background 10 μSv/day, chest X-ray 20 μSv, flight NY–LA 40 μSv, mammogram 400 μSv, chest CT 7 mSv (700 days), cancer risk from 100 mSv/yr, severe poisoning 2 Sv, often fatal 4 Sv, fatal 8 Sv • ★ **CT applications**: bones, fractures and trauma (fast); tumours and lungs; also angiography and surgery/radiotherapy planning • **MRI acquisition**: align hydrogen spins with the magnet → radio pulse → measure return to alignment → repeat over frequencies → reconstruct from **k-space** (Fourier domain); T1, FLAIR = contrast types • **why AI**: more images, higher resolution, faster scans, new modalities → anomaly detection, triage, reconstruction • **quantification vs decision**: AI should support clinicians, not replace them • planes: **sagittal** (left/right), **coronal/frontal** (front/back), **axial/transversal** (top/bottom) • **pixel** = picture element, **voxel** = volume element.

▷ ★ ***CT reconstruction*** **projection** = sum of intensities along parallel rays at one angle • **back-projection** = smear each ray-sum back over every pixel on its ray, add over all angles • few angles → streaks and blur, more angles → sharper (filtered back-projection removes the blur) • 2×2 image AB/CD: one equation per ray (rows $A{+}B$, $C{+}D$; columns $A{+}C$, $B{+}D$; diagonals $A{+}D$, $B{+}C$) → solve; plain back-projected value of a pixel = sum of all ray-sums through it.

L2 Segmentation I: notation & losses

segmentation = voxel-wise classification (classification: one label per image; detection: boxes) • pre-DL: thresholding, region growing, graph cuts, marching cubes → sensitive to parameters, hard to parallelise • DL: big data + GPUs → features are learned.

▷ ***Notation*** $\Omega \subset \mathbb{R}^D$ = image space • $x \in \mathbb{R}^{M\times\Omega}$ = image with M modalities (grey M=1, RGB M=3) • $\mathbb{K} = \{0{:}\,\textbf{background}, 1, \dots, K\}$, $|\mathbb{K}| = K{+}1$ (background kept apart → simpler notation) • $Y, S \in \mathbb{K}^\Omega$ = label, prediction (class numbers) • **one-hot**: $Y \mapsto y \in (\widehat{\Delta}^K)^\Omega \subset \{0,1\}^{(K+1)\times\Omega}$ = vertices of the simplex • **softmax**: $\mathcal{N}_\theta(x) \mapsto s_\theta \in (\Delta^K)^\Omega \subset [0,1]^{(K+1)\times\Omega}$ • **K-simplex** Δ^K = $K{+}1$ probabilities summing to 1 • ★ **argmax**: $s_\theta \mapsto S$ = for each voxel, the index of the class with the largest probability • **training** = $\min_\theta \mathcal{L}(s_\theta, y)$ by backprop (chain rule), $\theta_{t+1} = \theta_t - \gamma\nabla_\theta$ (γ = learning rate; SGD, Adam) • ★ **valid loss** = **real-valued + differentiable w.r.t. the network output (not w.r.t. the label; need not be in [0,1] or positive)**.

▷ ***Losses*** **cross-entropy** $\mathcal{L}_{CE} = \sum_{i\in\Omega}\sum_k y^{(i,k)}\big(-\log s_\theta^{(i,k)}\big)$ = per voxel, − log of the probability of its true class • ★ $\partial\mathcal{L}_{CE}/\partial s_\theta^{(i,k)} = -y^{(i,k)}/s_\theta^{(i,k)} = -1/s$ if y=1, else 0 • ★ **gradient → ∞ when $s \to 0$ for the true class** (confident and wrong) → fix: $\log(s+\epsilon)$, clamping • **Dice loss** $\mathcal{L}_{DSC} = \frac{1}{|\mathbb{K}|}\sum_k\big(1 - \frac{2I^{(k)}}{U^{(k)}}\big)$, $I = \sum_i y s_\theta$, $U = \sum_i (y + s_\theta)$ • $\partial\mathcal{L}_{DSC}/\partial s_\theta^{(i,k)} = -2(y^{(i,k)}U - I)/U^2 = -2(U-I)/U^2$ if y=1, $2I/U^2$ else • ★ **CE vs Dice gradient**: CE is **local** (only true-class voxels, each depends on its own probability), all voxels weigh the same → big classes dominate; Dice is **global** (depends on the whole region via I, U), non-zero for every voxel, normalised per class → robust to **class imbalance** • ★ **Dice NaN fix**: class empty in both y and $s \to 0/0 \to$ add ϵ to numerator and denominator (`num = 2*einsum(...) + eps, denom = einsum(...) + eps`) • **nnU-Net loss** (fixed) = $\mathcal{L}_{CE} + \lambda\mathcal{L}_{DSC}$ • ★ **why "entropy"**: von Neumann to Shannon: the name already exists in statistical mechanics, and nobody knows what entropy really is, so you always win the debate.

L4 Segmentation II: practice

FCN (fully convolutional network) = only size-preserving operations → any input size (segmentation, registration, reconstruction, denoising); a classic CNN has fixed input/output size • ★ **FCN, varying image sizes, batch 1: no resizing needed** (size only divisible by $2^{\text{nr. poolings}}$, else pad); batch > 1 OK too: pad/crop to equal size • **U-Net** = 2D encoder–decoder + skip connections, **V-Net** = 3D, **2.5D** = 2D convolutions with neighbouring slices as extra input channels → context; also DeepLab, ENet (project baseline) • **isotropic** = same voxel size on all axes, **anisotropic** = not (1×1×2 mm) → **resample** • **augmentation** = random flip, noise, rotation (geometric ones also applied to y) • **offline** = precomputed, dataset grows ×A • **online** = new random version each iteration, network never sees the same image twice • **sub-patching** = train on patches/2D slices when the scan is too big → network never sees the whole picture → **many patches empty for some classes** (watch loss corner cases) • ★ **post-processing**: **NMS / largest connected component** = label components, keep the biggest → only if exactly one object is expected (not for multiple lesions); **DenseCRF** = align the segmentation to image edges by minimising an energy: unary ($-\log s_\theta$) + pairwise kernels: **appearance** (distance + intensity → local consistency) and **smoothness** (distance → removes noise) → needs visible edges; **ensembling** = average several networks → better and more reliable, but costly.

▷ ***Weak supervision*** **full** = every voxel labelled; **weak** = some voxels or image-level: tags, dots, scribbles, **bounding box** (tells where the object is *not*, not where it is) • **proposal-based** (DeepCut) = make fake full masks, retrain, repeat → errors reinforce themselves • **direct loss** $\min_\theta \tilde{\mathcal{L}}(\tilde{y}, s_\theta) + \lambda\mathcal{R}(s_\theta)$ • **partial CE** = CE summed only over annotated voxels • regularisers: **CRF loss**, box tightness prior • **boundary loss** $\sum_i\sum_k s_\theta^{(i,k)}\phi^{(i,k)}$ with ϕ = signed distance to the label boundary → for very unbalanced tasks.

▷ ★ ***Hard structures (esophagus)*** **SegTHOR** = heart, aorta, esophagus, trachea in thoracic CT • esophagus: small (**class imbalance**), **low contrast** with surrounding tissue, variable shape • fixes: Dice/boundary loss or class weights, oversample patches containing it, 3D/2.5D context, crop around the region, post-processing, ensembling.

L5 Metrics I: Dice & reporting

choose per task (**Metrics Reloaded**) • **always use ≥ 2 metric families**: **overlap** (Dice), **distance** (Hausdorff), clinical • **Dice** $\text{DSC} = \frac{2|\Omega_y\cap\Omega_s|}{|\Omega_y|+|\Omega_s|} = \frac{2\sum ys}{\sum(y+s)} = \text{F1}$, unitless, on the hard segmentation • **Jaccard/IoU** $= \frac{|\cap|}{|\cup|} = \frac{\text{DSC}}{2-\text{DSC}}$ → same ranking as Dice • **Dice favours over-segmentation** (same nr. N of wrong voxels, G = ground truth: over $\frac{2|G|}{2|G|+N}$ > under $\frac{2(|G|-N)}{2|G|-N}$) • **Dice ignores shape and location** • ★ **corner cases**: $\text{DSC}(\emptyset,\emptyset) = 1$ (textbook gives 0/0), $\text{DSC}(\emptyset, Y) = \text{DSC}(S, \emptyset) = 0$ → fix: $+\epsilon$ in numerator and denominator → **bias**: inflates scores on empty samples, rewards predicting nothing → compute in 3D per patient, or exclude and report empty cases • report with and without post-processing • show distributions, not only the mean • **statistical testing**: define what one sample is; **slices or voxels as samples = p-hacking** • **inter-rater agreement** ≈90% DSC → labels are opinions.

L7 Metrics II: HD, resolution, calibration

significant figures: $7 \neq 7.00$; uncertainty (inter-rater, folds, runs) → 3–4 digits at most; 2 mm voxels → reporting 1.36 mm is meaningless • **accuracy** = close to the truth, **precision** = repeatable.

▷ ***Hausdorff distance*** **HD** = the maximum distance between two boundaries • **boundary** $\partial\Omega_S \subseteq \Omega_S$ • $\phi_A(b) = \min_{a\in A} d(a,b)$ • ★ $\text{HD}(S,Y) = \max\left\{\max_{i\in\partial\Omega_S}\min_{j\in\partial\Omega_Y} d(i,j), \max_{i\in\partial\Omega_Y}\min_{j\in\partial\Omega_S} d(i,j)\right\}$ • **HD95** = 95th percentile, **ASSD** = mean of the distances • ★ **steps** (per class, after one-hot): extract both boundaries → EDT on both boundaries → mask each EDT with the other boundary → reduce (max / 95th / mean); **upsampling is superfluous** • **boundary extraction** = convolve with a kernel that counts neighbours, keep voxels without a full neighbourhood (8-neighbour kernel includes diagonals, 4-neighbour does not) • ★ **boundary = whole set** when **objects are thin or tiny** (1–2 voxels wide, single voxels) or **connected only diagonally / touching the image border**.

▷ ***Runtime*** **EDT** (Euclidean distance transform; scipy on CPU, cuCIM on GPU) = slowest step, 2 per class • ★ **Dice needs 0 EDTs; HD and ASSD share them → $2K$ in total** (textbook, without reuse: $4K$) • ★ **speed-ups**: reuse EDTs, **skip classes missing from prediction or label**, crop to the bounding box, GPU (**DisTorch**) • **HD$(\emptyset,\emptyset) = 0$; one side empty → undefined** → default = scan diagonal in mm → biases against methods that miss objects • HD in **mm**: use voxel spacing (crucial for anisotropic scans) • tools disagree (>100 mm) • **a pixel is not a little square** = it is a point sample.

▷ ★ ***Wrong slice order (same in prediction and label)*** **Dice unchanged** (it only counts voxels) • **ASSD/HD change** (3D neighbourhoods broken).

▷ ***Calibration*** **calibration** = confidence matches accuracy (0.6 → right 60% of the time) • **ECE** = bin predictions by confidence, |accuracy − confidence| per bin, weighted mean; **nr. of bins is a hyperparameter**.

Course papers

▷ ***nnU-Net (Isensee 2021)*** **nnU-Net** = "no new U-Net", a self-configuring pipeline • **fixed parameters**: plain U-Net, CE + Dice, SGD, instance norm, deep supervision, augmentation • **rule-based parameters** from the ★ **dataset fingerprint** = image sizes, voxel spacing, modality, nr. classes, nr. training cases, foreground intensity mean/s.d./percentiles → resampling, intensity normalisation, **patch size, batch size and topology given GPU memory** • **empirical parameters** = choose 2D / 3D / 3D cascade (or ensemble) by 5-fold cross-validation, and whether to keep only the largest component • ★ **different GPU → can give a different "optimal" configuration**: GPU memory is an input to the rules, and non-deterministic training can flip the empirical choice.

▷ ***Loss Odyssey (Ma 2021)*** reviews and benchmarks 20 segmentation losses in 4 families: **distribution-based** (CE, weighted CE, TopK, focal), **region-based** (Dice, IoU, Tversky), **boundary-based** (boundary loss, HD loss), **compound** = weighted sums (e.g. Dice + CE) • **no single loss wins on every task** • **Dice-based compound losses are the most robust** choice • boundary

losses should be paired with a region loss.

▷ ***Metrics Reloaded (Maier-Hein 2024)*** framework (Delphi consensus) to pick metrics from a **problem fingerprint** = structured description of task, target structures, data and output • 4 problem categories: **image-level classification, object detection, semantic segmentation, instance segmentation** • 3 pitfall types: **wrong problem category** (detection framed as segmentation), **poor metric selection** (metric ignores task properties, e.g. small structures), **poor metric application** (e.g. wrong aggregation over patients/images).

▷ ***Implementation pitfalls of distance metrics (Podobnik & Vrtovec 2025)*** 11 open-source tools give different HD/HD95/ASSD/NSD for the same masks (>100 mm differences) → **a "significant improvement" can come from just switching tool** • 3 causes: **boundary extraction** method, **mathematical definitions** (percentiles, averaging), **edge cases** (empty masks); one tool even ignores voxel size • advice: follow Metrics Reloaded, **report tool name and version**, handle empty masks yourself, use correct units/spacing (Google DeepMind tool most accurate).

▷ ***DisTorch (Rony & Kervadec 2025)*** distance metrics are skipped because they are slow (minutes per 3D scan), complex and dependency-heavy • **DisTorch** = fast GPU Hausdorff: brute-force pairwise distances between boundary voxels (KeOps/Triton), skipping voxels shared by both boundaries → quadratic but fast on GPU; used in assignment 3.

▷ ***A pixel is not a little square (Smith 1995)*** **a pixel is a point sample**, not a small square (voxel: not a cube) • a continuous image is recovered from samples with a **reconstruction filter** • matters for distances, resampling and mm-level precision claims.

L3 Registration

registration = align a **moving** image $\mathcal{M}$ to a **fixed** image $\mathcal{F}$: $\min_{\mathcal{T}} \mathcal{L}(\mathcal{F}, \mathcal{T}(\mathcal{M}))$, $\mathcal{T} : \Omega_2 \to \Omega_1$ • **SSD** $\sum_i (\mathcal{F}^{(i)} - \mathcal{T}(\mathcal{M})^{(i)})^2$ = assumes identical intensities → same modality (CT–CT); **only SSD is not robust to intensity changes** • **NCC** = linear intensity relation → same modality with brightness change • **NGF** = gradient directions match → multi-modal edges • **MI** (mutual information) $= H(\mathcal{F}) + H(\mathcal{T}(\mathcal{M})) - H(\mathcal{F}, \mathcal{T}(\mathcal{M}))$ = statistical dependency → multi-modal (CT–MRI, PET–MRI) • **MIND** = self-similarity → very different modalities.

▷ ★ ***Affine matrix (rigid)*** **affine matrix** = 4×4 matrix in the file metadata (NIfTI/DICOM) mapping voxel coordinates → world/patient coordinates : $(x', y', z', 1)^\top = \begin{pmatrix} a_{11} & a_{12} & a_{13} & t_x \\ a_{21} & a_{22} & a_{23} & t_y \\ a_{31} & a_{32} & a_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} (x, y, z, 1)^\top$ (homogeneous coordinates) • encodes **voxel size** (scaling), **rotation**, **real-world position** (translation), shear; **not rigidity or elasticity** • translation = identity + last column (t_x, t_y, t_z); scaling = $\mathrm{diag}(a, b, c, 1)$; rotation = cos/sin block; shear = off-diagonal terms • combine $A = C \cdot R \cdot S \cdot T$ • rigid registration $\min_A \mathcal{L}(\mathcal{F}, \mathcal{T}_A(\mathcal{M}))$.

▷ ***Elastic*** **deformation field** = one vector per voxel, $\mathcal{T} : \Omega_2 \to \mathbb{R}^3$ • ill-posed → add a regulariser $\min \mathcal{L}(\mathcal{F}, \mathcal{T}(\mathcal{M})) + \lambda \mathcal{R}(\mathcal{T})$: smoothness (neighbours move alike), **no folds** (Jacobian > 0, ideally ≈ 1) • tool: elastix • simplify: rigid → affine → elastic, **pyramidal** (coarse → fine).

▷ ★ ***Rigid or non-rigid?*** **rigid**: adult bones (skull, femur, a single rib) • **non-rigid**: lungs (breathing), heart (beating), kidneys (soft, move with breathing), the ribcage as a whole • **teenagers scanned years apart: bones grow → affine/non-rigid** • patient moved → at least rigid.

▷ ***Learned (VoxelMorph)*** **VoxelMorph** = U-Net mapping $(\mathcal{F}, \mathcal{M})$ → deformation field • ★ **no labels needed**: train on image pairs with the registration similarity as the loss (unsupervised); segmentations/landmarks optional as extra supervision • ★ **segmentation U-Net → registration**: 2 input channels $(\mathcal{F}, \mathcal{M})$, 3 output channels (displacement), no softmax, add a **spatial transformer** to warp $\mathcal{M}$, loss = similarity + smoothness • ★ **classical** = best/correct result but slow (optimise every pair) • **learned** = fast inference but no correctness guarantee.

L6 Ethics, data, reproducibility

GDPR: consent, purpose, careful handling • ★ **retrospective studies** need consent for the new use; exceptions: **consent cannot be asked (e.g. deceased)** or cannot reasonably be required → **the ethics committee decides, not you** (NL: METC) • **DMP** (data management plan) before collection: security, quality, legal compliance • **data** = the image, **metadata** = name, date, sex, weight • ★ **pseudo-anonymised** = ID mapping kept somewhere; **anonymised** = mapping deleted • **re-identification** = by cross-referencing other information → mitigate by skull stripping/defacing, cropping; impossible for iris, whole body, body surface • ★ **ease of anonymisation**: biopsy scan < head MRI < whole-body CT < iris photo • **children ≠ small adults** • **CC licences**: BY = attribution, NC = non-commercial, SA = share alike, ND = no derivatives • **DTA** = data transfer agreement between institutions • safest data = data you don't have; breach → report ASAP • no patient data on GitHub; passphrases, password manager, 2FA, encrypted drives.

▷ ***Training without sharing data*** ★ **federated learning** = each site trains locally, sends weights (not images) • **differential privacy** • cost: ↓ performance, ↑ compute • ★ **FL underperforms → heterogeneous data (scanners, populations)**.

▷ ***Reproducibility*** ★ meanings: bit-identical weights / retrain on another machine / with other package versions / results within a confidence interval • randomness: initialisation, batch sampling, non-deterministic GPU operations → fix **seeds** (`random.seed`, `np.random.seed`, `torch.manual_seed`), `torch.use_deterministic_algorithms`, pinned versions, shared code and model, several runs • ★ **public code + private data = only partially reproducible** • ★ **public challenges**: same data, metrics and hidden test set → fair comparison; open access to data.

▷ ***Deployment*** radiologists with AI beat those without; AI won't replace them (**Hinton 2016 "stop training radiologists" was wrong**) • who is responsible? data collectors, engineers, company, regulator or user: unclear • automation → loss of intuition; confirmation bias • also fairness, robustness, interpretability.

L8 Reconstruction

undersampled k-space → faster MRI but artefacts • **compressed sensing** = classical, iterative • **learned reconstruction**: Ξ = spectral domain, $\mathsf{F} : \Xi \to \Omega$ Fourier transform, k-space $\mu_n = \mathsf{F}^{-1}(x_n)$; fully sampled scans, simulate undersampling, network predicts the image, $\mathcal{L}_{rec} = \sum_i (\tilde{x}_\theta^{(i)} - x^{(i)})^2$ (≈ SSD) • **VORTEX** = consistency loss between differently augmented undersamplings • same idea: denoising, super-resolution, modality translation, anomaly detection • limit: cannot recover information that was never measured.

L8 Foundation models & domain shift

foundation model = general model trained on big data; promptable? medical? fine-tuned? self-supervised? • ★ **SAM** = **prompt-based** (points, boxes, text), trained on 2D natural images; SAM2 video mode → 3D scan as video; **SAM3** also includes some public medical data • **MedSAM** = fine-tuned SAM • ★ **fine-tuning: ↑ in-domain, ↓ generalisation → on private CT, SAM beat MedSAM** • **nnInteractive** = natively 3D, more stable under prompt changes • **TotalSegmentator** = model (Apache) + dataset (CC-BY) • **Virchow** = histopathology model used for patch embeddings • ★ **domain shift** = trained on domain 1, used on domain 2; sources: scanner, vendor, protocol, operator, population • **domain adaptation** = source → target; **source-free** = source data unavailable • ★ fixes: **transfer learning** (fine-tune a pretrained model), domain adaptation, intensity normalisation, augmentation, multi-site data • ★ **US generalises worse than CT** (operator dependent) • ★ brain MRI → lung CT: fine-tune, don't retrain from scratch • ★ one big + two small hospitals: federated learning, or share weights and fine-tune locally.

L9 Classification

first ask the right question (binary, bins, regression?) • **label ≠ ground truth** = an opinion (inter-/intra-rater variability) • **Barrett's esophagus** panel = consensus if >75% agree • one network per rater → **virtual panel**: better, interpretable, better calibrated • **MIL** (multiple-instance learning) = whole-slide image as a bag of patches; positive bag = at least one positive patch • now: foundation-model embeddings + attention aggregator; one "smoking gun" patch (easy) vs combination of patches (hard) • **quantification > decision** • ★ **shortcut learning** (implants/contrast → "cancer") = model uses a spurious feature → fix: balance data, crop or segment the lung, check saliency maps, external validation.

Guest lectures

▷ ***Ben Haddou: cardiac MRI*** **cine MRI** = short-axis slices over the heartbeat; **LGE** = contrast showing scar • masks → **ejection fraction** • challenges: little data, arbitrary pose, subtle texture, motion • priors: periodic → phase on a circle (CyclePhase); continuous anatomy → **neural fields** (any resolution); pose → **SE(2)-equivariant** networks • **equivariant** = $f(g \cdot x) = g \cdot f(x)$ (segmentation); **invariant** = $f(g \cdot x) = f(x)$ (classification) • CNNs: translation only; vs rotation augmentation, which costs data • costs of equivariance: only the symmetries you build in, pooling/interpolation can break it, more memory, wrong prior hurts • priors help most with few labels.

▷ ***Bergman: drug discovery*** **AtomNet** = 3D CNN on voxelised protein + molecule • **random split → 99.9% via near-duplicates (data leakage)** → time split → **target split** by similarity = real performance • lesson: split by the unit you generalise to (patients) • publication bias → add **decoys** as negatives.

▷ ***Hagenauer: tumour clonality*** same origin (**metastasis**, palliative care) or **independent tumours** (curative)? sequencing ≈ EUR 1700 per tumour, >1 week; histology disagrees ≈22% • **false "clonal"** (precision) denies a cure, **false "independent"** (recall) misses spread • tile embeddings → **attention-MIL** • **Deep Gamblers** = extra **abstain** output, $\ell = -\log(p_y + p_{abstain}/o)$, $1 < o < 2$ (small o → abstaining cheap) → ≈98% precision on 20% answered, abstained cases go to sequencing • **shortcuts**: scanner, size, patient identity • AI solves the easiest question that fits the labels.

▷ ***Krap: pixels to patients*** Dice 0.94, so what? model ≈10% of a product; the rest is regulation, evidence, workflow, payment • FDA AI devices: 76% radiology, 96% cleared via 510(k) (equivalent to an existing device) • benchmark ≠ bedside • knee implant: CT under load → segment → register → measure motion • regulation: EU **MDR**, US **FDA**, **EU AI Act**; **intended use** sets the risk class • evidence: **technical, clinical, utility** • workflow beats accuracy • user ≠ payer ≠ saver.

green = key term • blue = formula • purple = example/number • **red = exam trap** • ★ = asked in a past exam

Study notes (one paragraph per lecture)

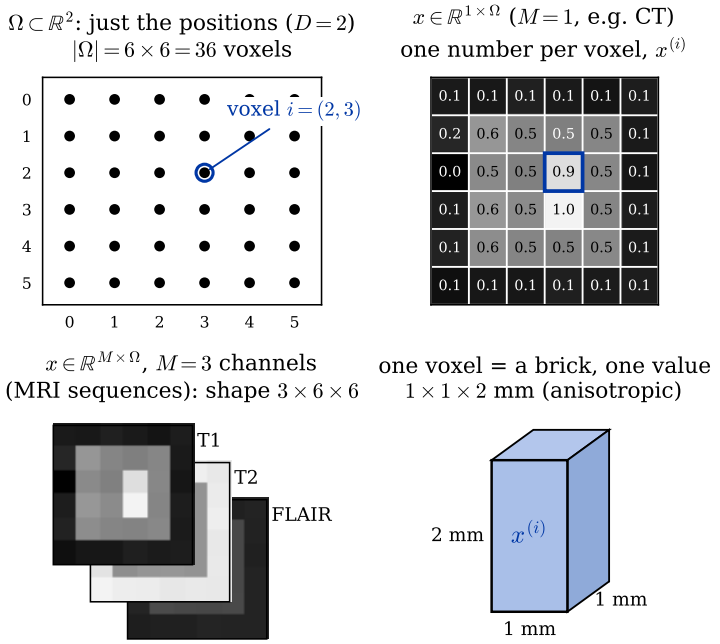
L1 Introduction: modalities

Medical imaging means looking inside the body without cutting it open. Each technology for doing this is called a **modality**, and each one trades off cost, speed, detail and safety, which is why hospitals keep all of them. The simplest is the **X-ray**: X-rays are shone through the body onto a detector, like a shadow puppet. Dense bone blocks the rays and looks white, while soft tissue lets most through and looks grey. You get a cheap, quick overview (is the arm broken?), but organs overlap into one flat picture, so you can't see fine organ detail, and the patient receives some radiation. **Ultrasound (US)** sends sound waves into the body and listens for the echoes, the same way a bat or a ship's sonar finds things. It is cheap, portable and safe (no radiation, no sedation), which is why it is used on pregnant women. Its weakness is that the image depends on how the operator holds and angles the probe. Two sonographers scanning the same patient produce different-looking images, so an AI trained on one hospital's ultrasounds **generalises worst to a new site**. Sound also can't pass through air or bone, so the lungs and the adult brain are hard to see (this is why gel is put on the skin: to remove the air gap). **CT (Computed Tomography)** spins the X-ray source around the patient and combines hundreds of shadow pictures into a 3D volume. Think of it as cutting a loaf of bread into slices without touching the bread. It is fast (seconds) and shows bones and organs well, so it is the go-to scan for **fractures and trauma**, **tumours and lungs**, blood vessels (angiography) and **planning surgery or radiotherapy**. Its intensities are also **quantitative**: on the **Hounsfield** scale air is about -1000, water is 0 and bone is several hundred or more, and the same tissue gets roughly the same number on every scanner. The cost is radiation, and sometimes an injected contrast agent. To see how CT turns shadows into slices, take a tiny 2×2 image with pixels A=1, B=2 (top row) and C=3, D=4 (bottom row). A **projection** is the sum along parallel rays at one angle: rows give A+B=3 and C+D=7, columns give A+C=4 and B+D=6, diagonals give A+D=5 and B+C=5. Each ray-sum is one equation, so with enough angles you can solve for the four unknowns. Plain **back-projection** doesn't solve the equations. It "smears" each ray-sum back over every pixel on that ray and adds them up, so pixel A gets 3+4+5=12, B gets 14, C gets 16 and D gets 18. That is exactly $2\times(\text{true value})+10$, where 10 is the sum of all pixels. The ordering is right, but every pixel carries the same extra "haze". That haze is the blur, and **filtered back-projection** is the trick that removes it. With only a few angles the smears don't cancel out and you see **streaks**; more angles give a sharper image. **MRI** uses a huge magnet and radio waves instead of radiation. The magnet lines up the body's hydrogen atoms (mostly in water) like compass needles. A radio pulse knocks them out of line, and the scanner records the signal they give off as they snap back. Different tissues snap back at different speeds, which is why MRI has the **best soft-tissue contrast** (brain, muscle, tumours) and can image any plane. Different timing recipes give different contrasts, such as **T1** or **FLAIR** (FLAIR darkens fluid so lesions stand out). The measurements are not pixels but frequencies, stored in **k-space** (the Fourier domain). Think of the image as a song and k-space as the list of notes: the centre of k-space holds the overall brightness and contrast, the edges hold the fine detail, and the image is **reconstructed** by adding the notes back together. Collecting all those notes takes a long time (often 20–60 minutes). That is why MRI is **expensive**, and why young children who can't lie still often need **sedation**. **PET (Positron Emission Tomography)** injects a radioactive tracer, typically radioactive glucose. Cells that use a lot of sugar, such as cancer cells, soak it up and glow, giving off gamma rays the scanner detects. PET therefore shows **function (what tissue is doing), not anatomy (what it looks like)**. Its images are blurry, so it is combined with CT or MRI (PET-CT) to show both where the glow is and what is there, at the cost of radiation. **PET is not "CT with contrast"**: a CT contrast agent (iodine) only blocks X-rays more, so vessels or organs look brighter (still anatomy), while in PET nothing shines through the body and the tracer itself emits the signal from inside (function). Grouping by risk: X-ray, CT and PET use **radiation**; US, MRI, endoscopy and histopathology don't. **Endoscopy** (a camera on a tube) and **histopathology** (a tissue sample cut out in a biopsy and looked at under a microscope) are **invasive**, while PET and contrast agents are only **minimally invasive** (a needle). Radiation dose is measured in **Sieverts (Sv)**, and a ladder of everyday numbers helps: eating a banana gives 0.1 μSv, an arm X-ray 1 μSv, daily natural background 10 μSv, a chest X-ray 20 μSv, a New York to Los Angeles flight 40 μSv, a mammogram 400 μSv, and a chest CT about 7 mSv. That chest CT is about 700 days of background radiation, or 350 chest X-rays. Cancer risk becomes measurable from about 100 mSv per year, and whole-body doses of 2, 4 and 8 Sv mean severe poisoning, often fatal and fatal respectively. That's why CT is used deliberately, not casually. **Why AI?** Hospitals produce more images, at higher resolution, from faster scans and new modalities, and the number of radiologists can't keep up. AI helps with **anomaly detection** (spot the nodule), **triage** (move the scan with a likely brain bleed to the top of the queue) and **reconstruction** (rebuild a good MRI from fewer k-space measurements, so scans get faster). The guiding principle is **quantification over decision**: AI should measure ("the tumour is 12 ml, up 20% since March") and support the clinician, who still makes the call ("operate"). AI is not meant to replace the clinician. Finally, the vocabulary for looking at a 3D scan: the **sagittal** plane splits the body into left and right (a side view, like a silhouette), the **coronal or frontal** plane splits front from back (like a headband crown over the ears), and the **axial or transversal** plane splits top from bottom (slicing the bread loaf from head to toe, the usual CT slice). A 2D image is made of **pixels** (picture elements), and a 3D scan is made of **voxels** (volume elements). For example, a CT might be 512×512×300 voxels, each a tiny 1×1×2 mm brick of tissue with one intensity value.

L2 Segmentation I: notation & losses

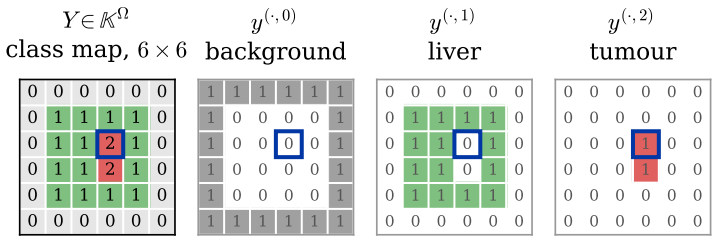
Three tasks ask increasingly precise questions about the same scan. **Classification** gives one label per image ("is there a tumour?" → yes), **detection** draws a box around each object ("it is roughly here"), and **segmentation** labels **every single voxel** ("these exact 1,243 voxels are tumour"). That is why segmentation is described as **voxel-wise classification**: it is a tiny classifier run at every voxel. Before deep learning, segmentation was hand-designed. **Thresholding** keeps every voxel above a value (CT voxels above ≈300 HU → bone). **Region growing** starts from a seed voxel and keeps adding neighbours with similar intensity, like the paint-bucket tool in Paint. **Graph cuts** split the image along the "cheapest" boundary, and **marching cubes** turns a finished mask into a 3D surface mesh. These methods break when you change their **parameters** (one threshold works for one scanner but not the next) and are hard to run in parallel. Deep learning replaced them because big datasets and GPUs let the network **learn the features** itself. The notation is just bookkeeping

for "an image, its labels and the network's guesses". $\Omega \subset \mathbb{R}^D$ is the grid of voxel positions, with D the number of dimensions (a 256×256 slice: D=2; a CT volume: D=3). The image $x \in \mathbb{R}^{M\times\Omega}$ stores M numbers at each position, one per channel (grey CT: M=1; RGB photo: M=3; brain MRI with T1, T2, FLAIR and T1-contrast: M=4).



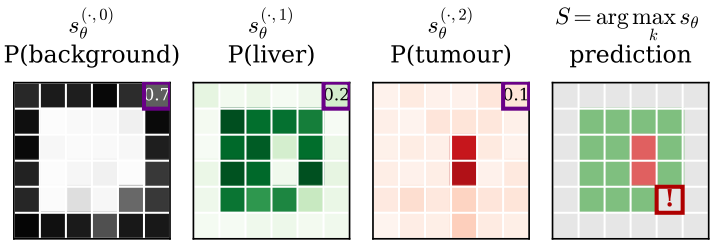
Ω is only the grid of positions; x puts M numbers on each position; in 3D each position is a small brick (voxel).

The classes are $\mathbb{K} = \{0, 1, \ldots, K\}$, where 0 is always **background**, so there are $|\mathbb{K}| = K+1$ classes in total (background, liver, tumour: K=2, $|\mathbb{K}|$=3). Keeping background separate simply makes the formulas tidier. The ground-truth label Y and the prediction S are maps that put one class number on each voxel ($Y = [0,0,1,1,2]$ for five voxels). For the maths it is easier to write each class number as a **one-hot** vector, with a 1 in its class's position and 0 elsewhere (class 1 out of $\{0,1,2\} \to [0,1,0]$); this gives y.



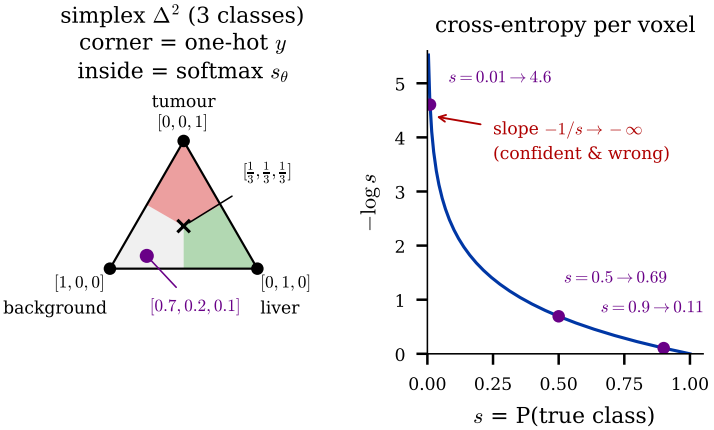
The label Y (one number per voxel, shape 6×6) becomes $K+1$ binary maps $y \in \{0,1\}^{(K+1)\times\Omega}$ (shape $3\times6\times6$). Each voxel has exactly one 1 across the maps: at the blue voxel Y=2 → $y^{(i)} = [0,0,1]$.

The network, however, does not output hard answers. Its last layer is a **softmax**, which turns raw scores into probabilities that are positive and sum to 1 at every voxel ($[0.7, 0.2, 0.1]$); this gives s_θ. The set of all such probability vectors is the **simplex** Δ^K. With 3 classes it is a triangle: a one-hot label is a **corner** (100% sure), a softmax output is a point **inside** (unsure), and the centre $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$ is maximally confused. To get the final segmentation, $\arg\max$ picks at each voxel the class with the highest probability ($[0.7, 0.2, 0.1] \to$ class 0), which gives S back.



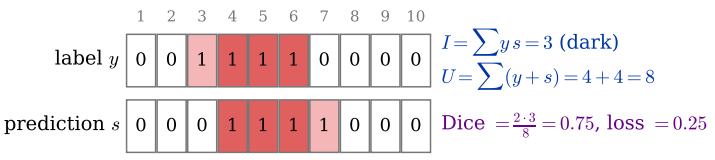
The network outputs one probability map per class, $s_\theta \in [0,1]^{3\times6\times6}$, summing to 1 at each voxel (purple voxel: $[0.7, 0.2, 0.1] \to$ class 0). Argmax collapses them into S; the red ! is a liver voxel the network got wrong.

Training means finding weights θ that make the loss $\mathcal{L}(s_\theta, y)$ small. Backpropagation (the chain rule) gives the gradient, the slope of the loss with respect to each weight, and **gradient descent** takes a step downhill: $\theta_{t+1} = \theta_t - \gamma\nabla_\theta\mathcal{L}$. Picture walking down a foggy hill by always stepping against the slope, with γ the **learning rate** setting the step length (θ=2, gradient 4, γ=0.1 → θ=1.6); SGD and Adam are smarter versions of this. Hence what makes a **valid loss**: it only has to be a **real number that is differentiable with respect to the network output**, because that derivative is what gets backpropagated. It does **not** need to be differentiable with respect to the label (labels are fixed, we never update them), and it does **not** need to lie in $[0,1]$ or be positive ($-$Dice, ranging from -1 to 0, is a perfectly good loss); only its slope matters. The standard loss is **cross-entropy**, $\mathcal{L}_{CE} = \sum_i \sum_k y^{(i,k)}\left(-\log s_\theta^{(i,k)}\right)$. Because y is one-hot, the inner sum just picks, at each voxel, $-\log$ of the probability given to the **true** class. Being confident and right costs little, and being confident and wrong costs a lot (true-class probability 0.9 → 0.11; 0.5 → 0.69; 0.01 → 4.6). Its gradient is $\partial\mathcal{L}_{CE}/\partial s^{(i,k)} = -y^{(i,k)}/s^{(i,k)}$: $-1/s$ for the true class and 0 for all others (s=0.9 → -1.1; s=0.01 → -100). As $s \to 0$ the gradient goes to $-\infty$, so a confidently wrong voxel can blow up training into NaNs; the fix is $\log(s+\epsilon)$ or clamping s away from 0.



Left: every softmax output lives in the triangle; argmax colours it by nearest corner. Right: CE explodes as the true-class probability goes to 0.

The **Dice loss** instead measures overlap per class: $\mathcal{L}_{DSC} = \frac{1}{|\mathbb{K}|}\sum_k\left(1 - \frac{2I^{(k)}}{U^{(k)}}\right)$, with intersection $I = \sum_i y\, s_\theta$ and "union" $U = \sum_i (y + s_\theta)$ (strictly, the two sizes added together). For example, take a 1D image where the tumour is voxels 3–6 and the prediction is voxels 4–7, both hard: $I = 3$ shared voxels, $U = 4 + 4 = 8$, so Dice $= 6/8 = 0.75$ and loss $= 0.25$.

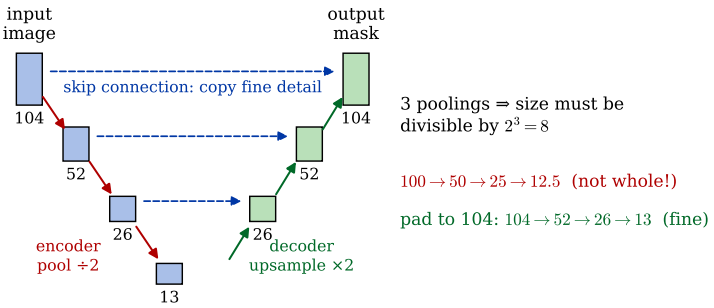


Dark = voxels in both label and prediction (intersection I); light = only in one.

The softmax version does the same sums with probabilities instead of 0/1. Its gradient is $-2(U-I)/U^2$ for voxels that belong to the class (negative, so training raises their probability) and $2I/U^2$ for voxels that don't (positive, so training lowers it); with I=3, U=8: -0.16 and $+0.09$. The two losses behave differently. CE is **local**: each voxel's gradient depends only on its own probability, and every voxel counts equally, so the class with the most voxels dominates. Dice is **global**: each gradient depends on the whole region through I and U, every voxel gets a non-zero push, and every class is normalised by its own size. This matters for **class imbalance**. Picture a scan of 1,000,000 voxels with only 100 tumour voxels: predicting "all background" gets 99.99% of voxels right, so CE barely complains, but the tumour Dice is 0, which costs that class its full loss of 1 and adds 0.5 to the 2-class average. Dice has its own trap. If a class is **absent from both label and prediction** (a slice with no tumour), then $I = U = 0$ and Dice is $0/0$ = **NaN**. The fix adds a small ϵ to both: $\frac{2I+\epsilon}{U+\epsilon}$, so an empty-and-correctly-empty class scores $\epsilon/\epsilon = 1$ (in code: `num = 2*einsum(...) + eps, denom = einsum(...) + eps`). Since each loss covers the other's weakness, **nnU-Net** simply uses both: $\mathcal{L}_{CE} + \lambda\mathcal{L}_{DSC}$. And a story worth remembering for the exam: when Shannon asked what to call his new measure of information, von Neumann reportedly told him to call it **entropy**, because the name already existed in physics and nobody really knows what entropy is, so in a debate you will always have the advantage.

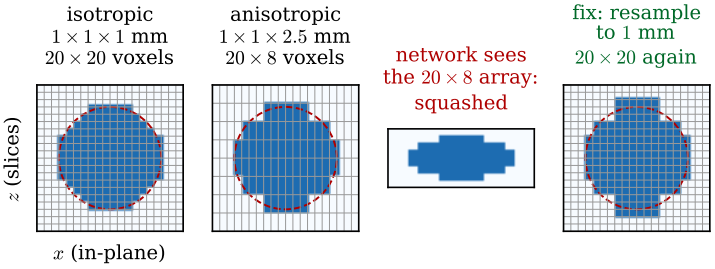
L4 Segmentation II: practice

L2 was about the loss. L4 covers everything else that makes a segmentation network work on real scans. A classic CNN classifier ends in fully connected layers that expect a fixed number of inputs: train it on 256×256 images and it cannot take a 300×300 image without resizing. A **fully convolutional network (FCN)** uses only operations that slide over the image (convolution, pooling, upsampling), so it accepts **any input size** and returns a map of the same size. That makes it the natural tool for every image-to-image task: segmentation, registration, reconstruction and denoising. Hence the exam trap: **an FCN trained with varying image sizes and batch size 1 needs no resizing**. The only constraint is that each pooling halves the size, so with p poolings the size must be divisible by 2^p; otherwise you **pad** (3 poolings, 100 px: $100 \to 50 \to 25 \to 12.5$ does not work; pad to 104: $104 \to 52 \to 26 \to 13$ works). Batch size 1 is not a requirement, it just makes things trivial: a batch is one tensor of shape (B, C, H, W), so all images *within one batch* must have the same size. With B=1 every batch holds a single image and sizes can change freely between iterations. With B>1 it works too, as long as you **pad** the images to the largest in the batch or **crop** fixed-size patches (what nnU-Net does). Still **no resizing**: padding and cropping keep the voxel spacing. The **U-Net** is the standard FCN. Its **encoder** shrinks the image to understand *what* is there, its **decoder** grows it back to say *where*, and **skip connections** copy fine detail from the encoder straight to the decoder so boundaries stay sharp.

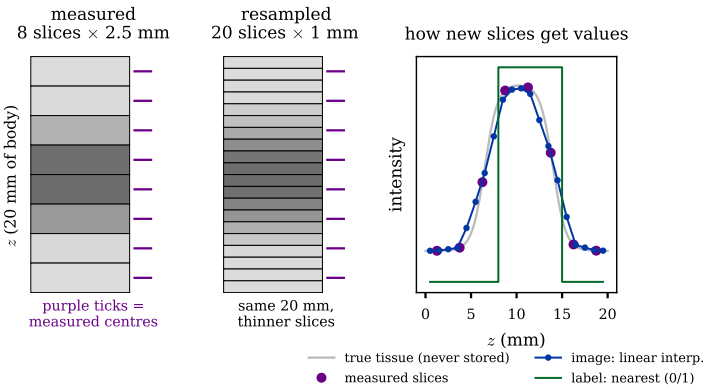


U-Net: down the left side (red, pooling), up the right side (green, upsampling), skip connections across (blue). Each pooling halves the size, hence the divisibility rule.

Before training, look at the voxel size. **Isotropic** voxels are equal on every axis, little cubes (1×1×1 mm). **Anisotropic** voxels are not: CT and MRI are usually sharp within a slice but have thick slices, so the voxels are tall bricks (1×1×2.5 mm). The network only sees the array of numbers, not the millimetres, so to it a round organ stored with thick slices looks **squashed**. The **fix** is to **resample** every scan to a common spacing before training: interpolate new slices in between (linear or B-spline for the image, **nearest neighbour for the label**, so it keeps whole class numbers instead of inventing 0.5s). Resampling restores the shape, but **not the detail that was never measured** between the thick slices.

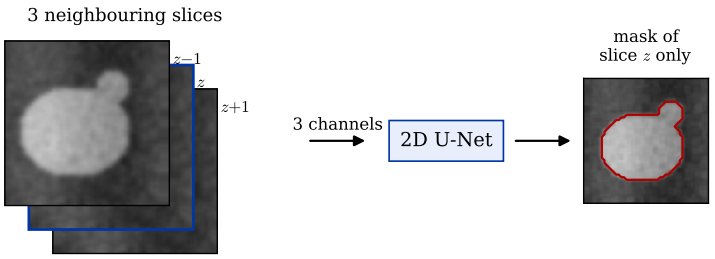


Side view through a round organ (dashed red = true outline in mm). Isotropic: square voxels. Anisotropic: tall bricks, only 8 rows along z. The network sees the raw 20×8 array, squashed. Fix: resample to 1 mm, 20 × 20 and round again, but blocky along z.

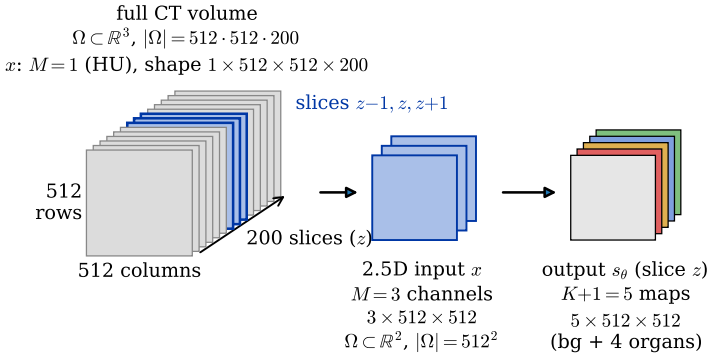


What resampling does to the stack: the same 20 mm of body is cut into 20 thin slices instead of 8 thick ones. Each new slice gets its value by interpolating between the two nearest measured slices (blue: linear for the image; green: nearest neighbour for the label, so it stays 0/1). The new slices are smoother, not sharper: the true tissue curve (grey) between measured slices is never recovered.

The **V-Net** is the same idea as the U-Net in 3D: it sees full 3D context but needs much more GPU memory. **2.5D** is the compromise: a 2D network that receives the neighbouring slices $z-1$, z, $z+1$ as 3 input channels and segments slice z, giving some 3D context at 2D cost. DeepLab and ENet (lightweight, the project baseline) are other architectures.



2.5D: slices $z-1, z, z+1$ go in as 3 channels of a 2D U-Net, which outputs the mask of the middle slice z only.



The same CT seen with L2 notation: the full 3D volume, the 3-slice 2.5D input, and the network's output for the middle slice.

Notation, worked CT example

Where 512×512×200 comes from. A CT scan is a stack of 2D slices. Each slice is an image of 512×512 pixels (512 rows × 512 columns, the standard CT image size), and the scanner takes 200 such slices along the body (the z axis, head to feet). So the volume is 512×512×200: rows × columns × slices.

Ω and $|\Omega|$. Ω is the set of all voxel *positions*, without their values. Each position has D=3 coordinates (row, column, slice), so $\Omega \subset \mathbb{R}^3$. The bars $|\cdot|$ mean "how many": $|\Omega|$ is the *number* of voxels, $512 \cdot 512 \cdot 200 \approx 52$ million.

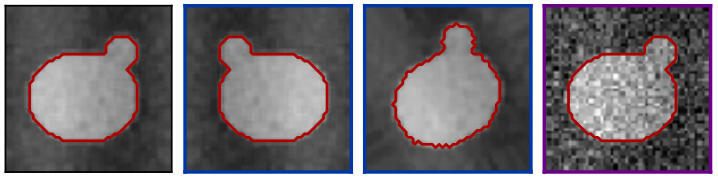
M is the number of values stored at each position (the channels). CT measures one thing per voxel, X-ray attenuation in HU, so M=1 and $x \in \mathbb{R}^{1\times\Omega}$, shape 1×512×512×200. (An RGB photo has M=3; a brain MRI with T1, T2, FLAIR, T1c has M=4.)

2.5D input. The network segments one slice z at a time, so its Ω is just that slice: $\Omega \subset \mathbb{R}^2$, $|\Omega| = 512 \cdot 512 \approx 262{,}000$. It receives slices $z-1, z, z+1$ stacked as channels, so M=3 and x has shape 3×512×512. **Here M=3 is not three kinds of measurement**: it is the same CT, three neighbouring slices.

Output. With SegTHOR's 4 organs plus background, K=4, so $s_\theta \in [0,1]^{(K+1)\times\Omega}$ is 5 probability maps of 512×512, for slice z only.

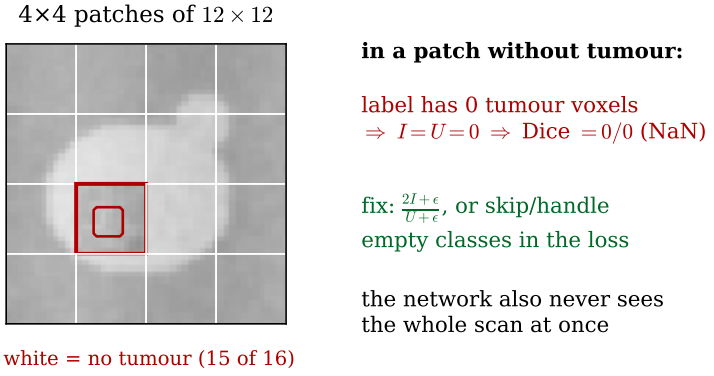
Voxel size. With spacing 1×1×2.5 mm, a slice covers 512 mm and 200 slices cover $200 \times 2.5 = 500$ mm of body. Resampling to 1 mm isotropic keeps the 500 mm but needs 500 slices, giving 512×512×500.

Medical datasets are small, so we **augment** them by creating realistic variations of each training image. **Geometric** transforms (flip, rotation, scaling) move voxels around, so **the same transform must be applied to the label y**, or the mask would point at the wrong place. **Intensity** transforms (noise, brightness) leave every voxel where it is, so the label stays unchanged.



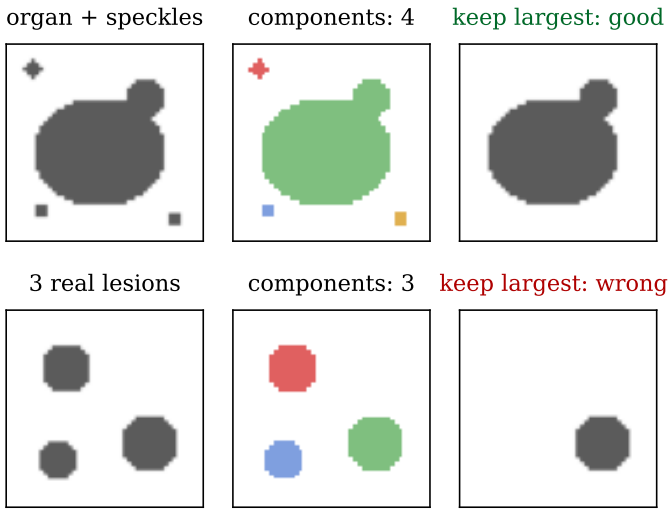
Blue frames: geometric, applied to image and mask. Purple frame: intensity, image only, mask unchanged.

Offline augmentation precomputes A versions of every image and saves them, so the dataset grows $\times A$ on disk but is still finite. **Online** augmentation draws a new random transform at every iteration, so the network never sees exactly the same image twice and nothing extra is stored. A full 3D CT (512×512×200) often does not fit in GPU memory, so we use **sub-patching**: train on random patches or 2D slices. The cost is that the network never sees the whole picture, and that **many patches contain no voxel at all of a small class**. Those are exactly the empty-class corner cases from L2 (Dice 0/0), so the loss must handle them.



A small tumour falls into only one of 16 patches; every other patch has an empty tumour class.

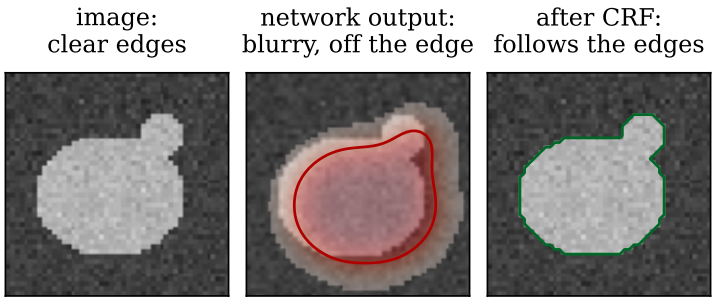
After prediction, **post-processing** cleans up the output. **Largest connected component** (also listed as NMS) groups touching foreground voxels into components and keeps only the biggest. This removes small speckle false positives far from the organ, but it is **only valid when exactly one object is expected** (one liver, one heart): with multiple lesions or metastases it deletes real findings.



fine when exactly one object is expected; deletes real lesions otherwise

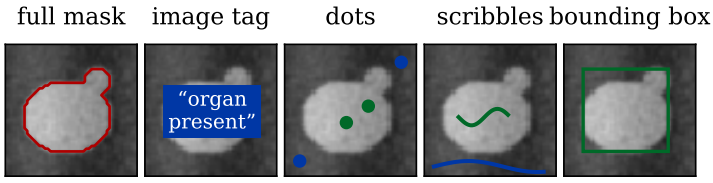
Top: one organ plus 3 false speckles, so keeping the largest component is right. Bottom: 3 real lesions, so keeping the largest silently deletes two of them.

DenseCRF refines the mask by minimising an energy, i.e. a total cost. The **unary** term $-\log s_\theta$ is the cost of giving a voxel a label the network found unlikely, so it keeps the result close to the network's opinion. The **pairwise** terms charge a cost whenever two voxels get different labels. The **appearance kernel** applies when the voxels are close *and* similar in intensity, so labels spread along regions of similar brightness and stop at edges (local consistency). The **smoothness kernel** applies when they are simply close, which removes isolated noisy voxels. Because it relies on intensity differences, a **CRF needs visible edges**: it cannot help where an organ has the same brightness as its surroundings.



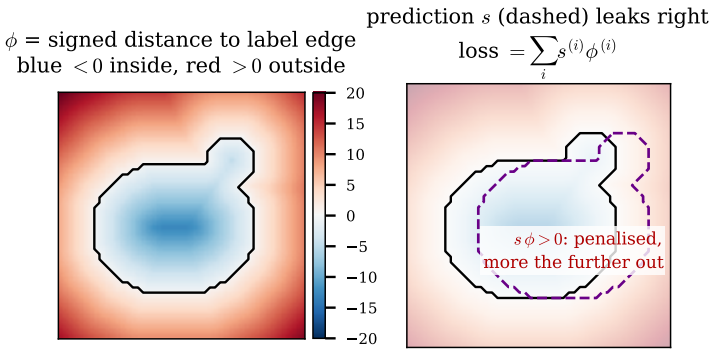
Idealised DenseCRF effect: the blurry, shifted network output (middle) is pulled onto the visible image edges (right).

Ensembling trains several networks (different seeds, folds or architectures) and averages their softmax outputs. Their individual mistakes partly cancel, like asking several doctors instead of one, so results are better and more reliable, at the cost of $N\times$ the training and inference compute. Full voxel labels are expensive (a radiologist can spend hours on one 3D scan), so **weak supervision** learns from cheaper annotations: image-level **tags**, **dots**, **scribbles** and **bounding boxes**. Note that **a box mainly tells you where the object is *not***: everything outside is surely background, but inside is a mix of object and background.



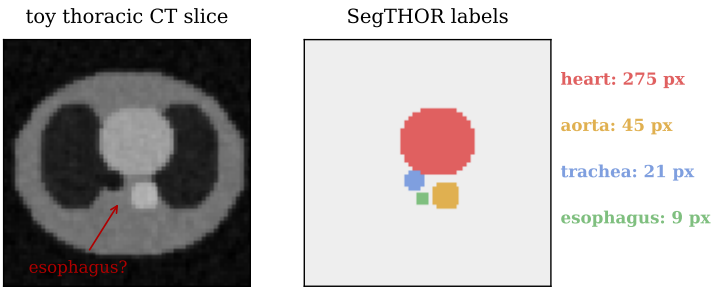
A full mask next to four cheaper weak labels (green = organ, blue = background). For a box, only "outside = background" is certain.

There are two ways to train with weak labels. **Proposal-based** methods (**DeepCut**) turn the weak labels into fake full masks (everything inside the box = organ), train on them, use the network's predictions as the next masks, and repeat. The danger is that **early mistakes are learnt and reinforced** in every round. **Direct-loss** methods train on the weak labels as they are: $\min_\theta \widetilde{\mathcal{L}}(\tilde{y}, s_\theta) + \lambda\mathcal{R}(s_\theta)$. The first term uses only what is known. With **partial cross-entropy**, CE is summed over the annotated voxels only (the scribble pixels), and unlabelled voxels contribute nothing. The regulariser $\mathcal{R}$ fills in the rest with prior knowledge. A **CRF loss** asks that nearby voxels with similar intensity share a label. A **box tightness prior** asks that the object touch every side of its box, so every row and column crossing the box should contain some object. Finally, the **boundary loss** $\sum_i \sum_k s_\theta^{(i,k)} \phi^{(i,k)}$ uses a signed distance map ϕ to the label boundary: negative inside the object, positive outside, growing with distance. Probability placed outside therefore costs more the further away it is, and probability inside lowers the loss. Because it measures distance rather than counting region voxels, it suits **very unbalanced** tasks, and it is usually paired with a region loss such as Dice.



Left: ϕ is negative (blue) inside the label and positive (red) outside. Right: a prediction leaking to the right pays $s\,\phi > 0$ for every leaked voxel, and more the further out it leaks.

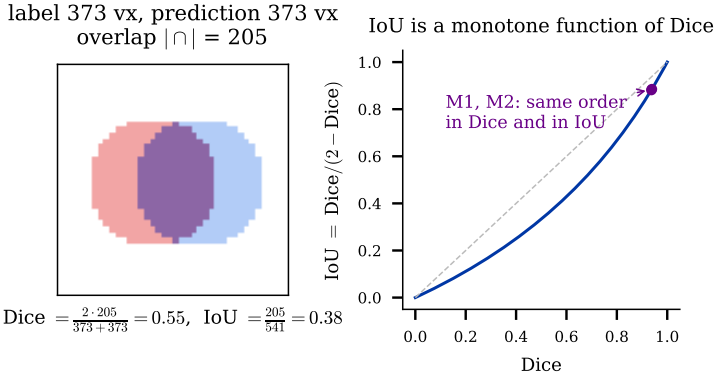
All of this comes together on a hard structure. **SegTHOR** asks for the heart, aorta, esophagus and trachea in thoracic CT, and the **esophagus** is the hardest of the four. It is **small** (strong class imbalance), it has **low contrast** with the surrounding tissue (unlike the dark, air-filled trachea or the bright aorta), and its **shape varies** (collapsed, or containing air). Fixes come straight from this lecture: Dice or boundary loss or class weights against imbalance, **oversampling patches** that contain it, **3D/2.5D context** to follow the tube from slice to slice, **cropping** around the region, post-processing (there is exactly one esophagus, so keeping the largest component is valid), and ensembling.



Toy slice: the esophagus is tiny next to the heart and barely differs in brightness from its surroundings, so it is both imbalanced and low-contrast.

L5 Metrics I: Dice & reporting

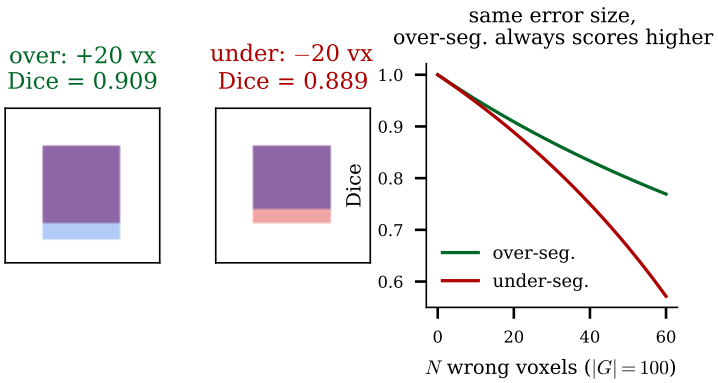
A metric is only useful if it answers the question the task asks, so the choice **depends on the task** (classification: accuracy, sensitivity; localisation: IoU, missed boxes; segmentation: Dice, Hausdorff) and on the problem itself: one class or many, one object or a variable number of them, compact or thin shapes. **Metrics Reloaded** turns this into a structured choice, and its main rule is **always use at least 2 metric families**: **overlap** (Dice), **distance** (Hausdorff) and, when relevant, a **clinical** measure (ejection fraction, tumour volume). (Also: always look at your data first; broken labels are common.) The workhorse is the **Dice score**, computed per class on the *hard* segmentation (after argmax, not on probabilities): $\mathrm{DSC} = \frac{2|\Omega_y \cap \Omega_s|}{|\Omega_y|+|\Omega_s|} = \frac{2\sum ys}{\sum(y+s)}$, i.e. twice the overlap divided by the two sizes added up. It equals the F1 score, is **unitless**, and goes from 0 (no overlap) to 1 (identical). The **Jaccard index / IoU** $\frac{|\cap|}{|\cup|}$ looks different but is a fixed function of Dice, $\mathrm{IoU} = \frac{\mathrm{DSC}}{2-\mathrm{DSC}}$, which only ever increases with Dice. So **IoU always ranks methods exactly as Dice does**, and reporting both adds no information (M1: Dice 0.9369 → IoU 0.881; M2: 0.9391 → 0.885, same order).



Left: label (red) and prediction (blue), overlap in purple; Dice counts the overlap twice against both sizes, IoU against the union. Right: IoU is a monotone function of Dice, so rankings never change.

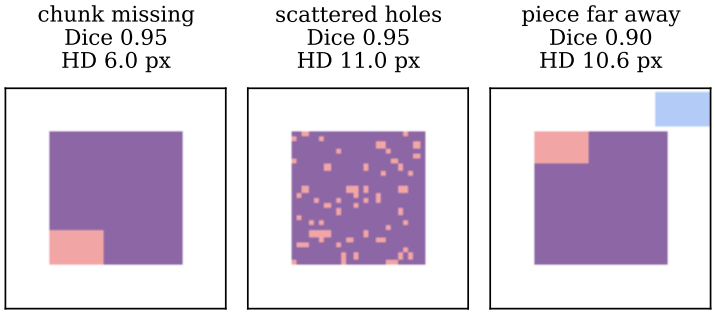
Dice has known biases. It **favours over-segmentation**: take a perfect prediction of an object with $|G| = 100$ voxels and

make $N = 20$ voxels wrong. Adding them outside gives $\frac{2|G|}{2|G|+N}$ $= 200/220 = 0.909$, removing them inside gives $\frac{2(|G|-N)}{2|G|-N}$ $= 160/180 = 0.889$. Same number of mistakes, but painting too much is punished less than painting too little, for every $N > 0$.



Same 20 wrong voxels: over-segmentation (blue strip outside) keeps a higher Dice than under-segmentation (red strip missed). The gap widens as errors grow.

Dice also **ignores shape and location**: it only counts voxels, not *where* the errors are. A missing chunk, scattered holes, or a piece predicted far away can give the same Dice while being very different clinically; a distance metric (L7) tells them apart.



Three predictions of the same square (red = missed, blue = extra). The first two have identical Dice (0.95) but different Hausdorff distances; the third puts a block far away with almost the same Dice.

The **corner cases** need a decision too. With something to predict and nothing predicted (or the reverse), Dice is 0. With **nothing to predict and nothing predicted**, the textbook formula gives $0/0$, although the answer is perfect. Implementations usually return 1, or add ϵ to numerator and denominator, $\frac{2I+\epsilon}{U+\epsilon}$, which gives $\epsilon/\epsilon = 1$. That introduces a **bias**: every empty slice or patch scores a perfect 1, which inflates averages and rewards a model that simply predicts nothing on hard cases. Better: compute Dice in **3D per patient** (where an organ is almost never absent), or exclude empty cases and report them separately.

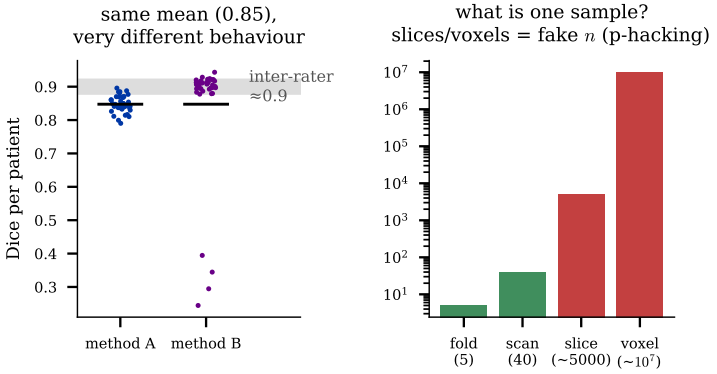
Dice corner cases

	predicted something	predicted nothing
something to predict	normal (0 to 1)	0
nothing to predict	0	$0/0$ = ? $+\epsilon \to 1$

fix: $(2I+\epsilon)/(U+\epsilon)$; biases scores up on empty cases

The four Dice cases: only "nothing to predict, nothing predicted" is undefined, and the ϵ fix turns it into a perfect score.

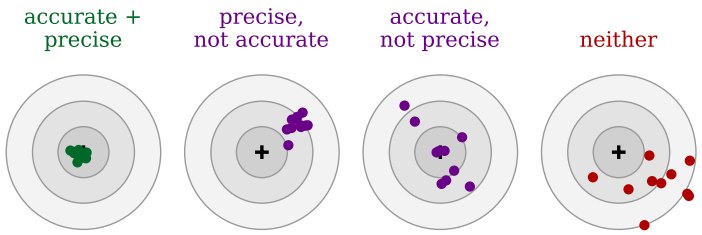
How results are reported matters as much as the metric. Report metrics **with and without post-processing** (with shows the best achievable result, without shows what the method itself does). **Show distributions, not only the mean**: two methods with the same mean Dice can be one that is always decent and one that is usually excellent but sometimes fails completely, and that difference matters for patients. For **statistical testing**, first define what one sample is: a fold, or a patient/scan, are honest units. **Treating slices or voxels as samples is p-hacking**: neighbouring slices of the same patient are not independent, so you inflate n from 40 to thousands and almost any difference becomes "significant". Finally, keep the **inter-rater agreement** in mind: two expert annotators typically agree at about 90% Dice, so labels are opinions, and a 0.2-point Dice difference above that level is probably noise.



Left: same mean Dice, very different behaviour; method B fails badly on four patients. The grey band marks human inter-rater agreement. Right: the number of "samples" explodes if slices or voxels are counted as independent.

L7 Metrics II: HD, resolution, calibration

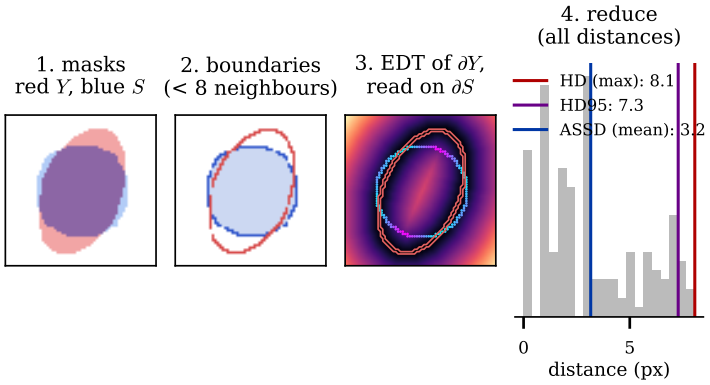
Every reported number carries a claim about its precision. **Significant figures**: writing 7 means "somewhere between 6.5 and 7.5", writing 7.00 claims you know it to the hundredth. Since results vary with the annotator (inter-rater), the folds and the training runs, **3–4 significant digits is the most that makes sense**. Resolution sets a limit too: with 2 mm voxels, reporting an average distance of 1.36 mm claims more than the scan can see (and, recall, a pixel is a point sample, not a little square). Keep two words apart: **accuracy** = close to the true value, **precision** = repeatable (the results agree with each other).



Accuracy is about the centre of the cloud, precision about its spread. A biased but repeatable method is precise, not accurate.

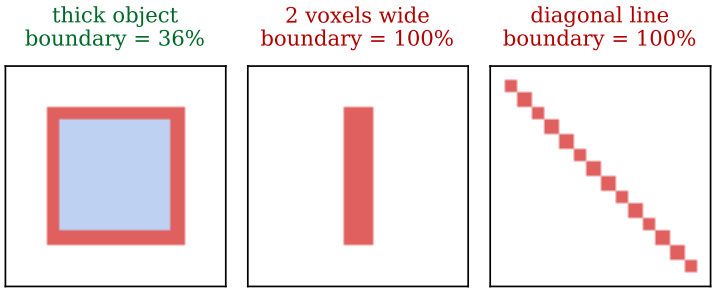
The **Hausdorff distance (HD)** is "the largest distance between two boundaries": how far the worst part of the prediction is from the label. For a voxel b and a set A, let $\phi_A(b) = \min_{a \in A} d(a, b)$ be the distance from b to the closest point of A. Then $\mathrm{HD}(S, Y) = \max\{\max_{i \in \partial\Omega_S} \phi_{\partial\Omega_Y}(i),\ \max_{i \in \partial\Omega_Y} \phi_{\partial\Omega_S}(i)\}$: for every boundary point of the prediction find its nearest label-boundary point, keep the worst, do the same the other way round, and take the larger of the two. Both directions are needed (a small prediction inside a big organ is close to the label from its own point of view, but the far side of the organ is far from it). Taking the **95th percentile** instead of the max gives **HD95**, which ignores a few outlier voxels; the **mean** gives **ASSD**. In practice, per class after one-hot: (1)

extract the two boundaries by convolving with a kernel that counts neighbours and keeping object voxels without a full neighbourhood (an 8-neighbour kernel includes diagonals, a 4-neighbour one does not); (2) compute the **EDT** (Euclidean distance transform) of each boundary, a map giving every voxel's distance to that boundary; (3) **mask** each EDT with the other boundary; (4) **reduce** with max, 95th percentile or mean. **Upsampling is superfluous**: it adds compute but no information.



The lecture example (circle S vs ellipse Y): masks, boundaries, the distance map of ∂Y read off at the points of ∂S, and the three ways to reduce all those distances.

For **thin or tiny objects** (1–2 voxels wide, single voxels) and objects **connected only diagonally or touching the image border**, every voxel lacks a full neighbourhood, so the **boundary is the whole set**.



Red = boundary voxels. For a thick object the boundary is a thin ring; for a 2-voxel-wide bar or a diagonal line it is the entire object.

Runtime: the EDT is the slow step (CPU, scipy; GPU, cuCIM), and you need 2 per class. **Dice needs 0 EDTs, and HD and ASSD can share the same two**, so $2K$ EDTs in total for K classes (computing each metric separately, the textbook way, costs $4K$). Speed-ups: reuse the EDTs, **skip classes missing from prediction or label**, crop to the object's bounding box, or use the GPU (**DisTorch**). Corner cases again: **HD**$(\emptyset, \emptyset) = 0$, but if only one side is empty the distance is **undefined**. A common default is the scan diagonal in mm, the largest possible distance, which **heavily penalises methods that miss objects**. HD has units, so compute it in **mm using the voxel spacing**, crucial for anisotropic scans (a 1-voxel step along z may be 2.5 mm, not 1). Different tools can disagree by >100 mm on the same masks, so report which one you used.

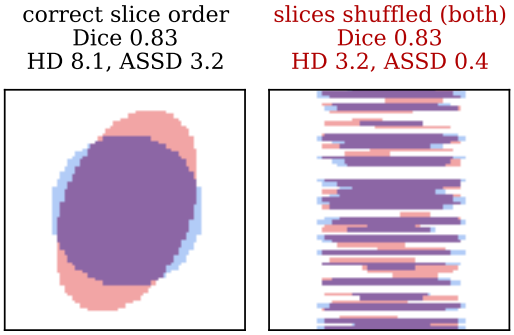
Hausdorff corner cases

	predicted something	predicted nothing
something to predict	normal (mm)	undefined
nothing to predict	undefined	0

default for undefined: scan diagonal (mm)

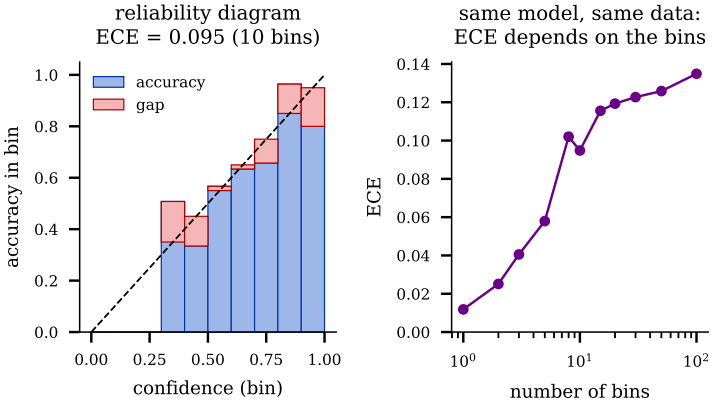
Hausdorff corner cases: unlike Dice, the one-side-empty cases are undefined, and the convention you pick changes the averages.

A diagnostic trick: if the **slice order is wrong, but the same in prediction and label**, **Dice is unchanged** (it only counts voxels) while **HD and ASSD change** (3D neighbourhoods are broken, so boundaries and distances are different).



Shuffling the rows of label and prediction identically keeps every voxel's overlap (Dice 0.83 both times) but changes the boundaries, so HD and ASSD change completely.

Finally, **calibration**: a model is calibrated when its confidence matches its accuracy (among all predictions made with 60% confidence, about 60% are right). The **ECE** (expected calibration error) puts predictions into confidence **bins**, computes in each bin the gap |accuracy − confidence|, and averages the gaps weighted by how many predictions each bin holds: ECE $= \sum_b \frac{|B_b|}{n} |\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$. **The number of bins is a hyperparameter**: few bins average over- and under-confidence away, many bins add noise, so ECE values are only comparable with the same binning.

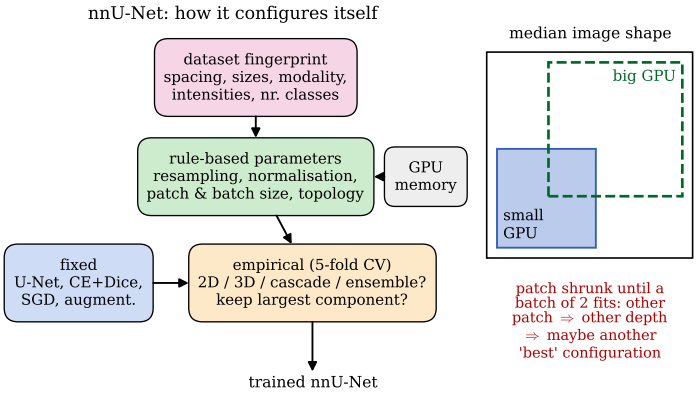


Left: reliability diagram; bars below the diagonal are overconfident, above it underconfident. Right: the same predictions give an ECE between 0.01 and 0.14 depending only on the number of bins.

Course papers

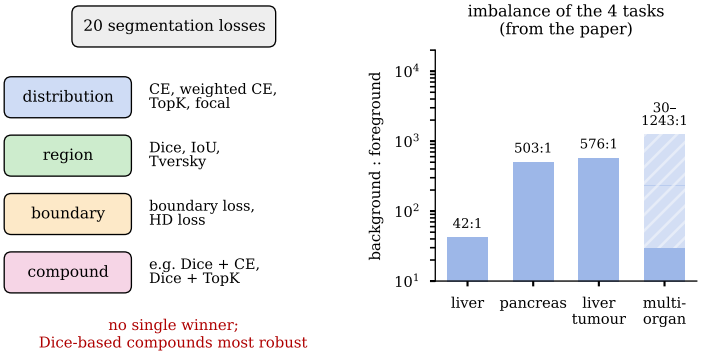
The six course papers form one story: how to *build* a good segmentation method (nnU-Net, Loss Odyssey), how to *measure* it honestly (Metrics Reloaded, implementation pitfalls, DisTorch), and what a pixel actually *is* (Smith). **nnU-Net** (Isensee 2021, "no new U-Net") starts from an uncomfortable observation: a plain U-Net that is *configured* well beats most fancy new architectures, but configuring it by hand takes expert trial and error that rarely transfers between datasets. So nnU-Net configures itself, splitting every design decision into three kinds. **Fixed parameters** are the same for every dataset: a U-Net-like encoder–decoder with instance normalisation (each feature map of each image is rescaled to mean 0, s.d. 1 on its own, so it works even with tiny batches) and deep supervision, CE + Dice loss, SGD with Nesterov momentum, a fixed augmentation set, and 1000 epochs with foreground oversampling. **Rule-based parameters** are computed from the **dataset fingerprint**, a standard description of the data: image sizes, voxel spacing, modality, intensity distribution (fore-

ground mean, s.d., percentiles), number of classes and number of training cases. Heuristic rules turn this fingerprint into resampling, intensity normalisation and the network shape. For example, CT is clipped to dataset percentiles and z-scored with the global foreground mean and s.d. (mean 100 HU, s.d. 50: a 200 HU voxel becomes $(200-100)/50=2$), while MRI is z-scored per image. For the network shape, the patch starts at the median image shape and is shrunk, while the topology is adapted, until a batch of at least 2 fits in GPU memory. **Empirical parameters** are the few choices that are simply tried with 5-fold cross-validation: 2D U-Net, 3D U-Net, or 3D cascade (a low-resolution 3D net first, then a full-resolution refinement), or an ensemble of them; and whether keeping only the largest connected component improves the score.



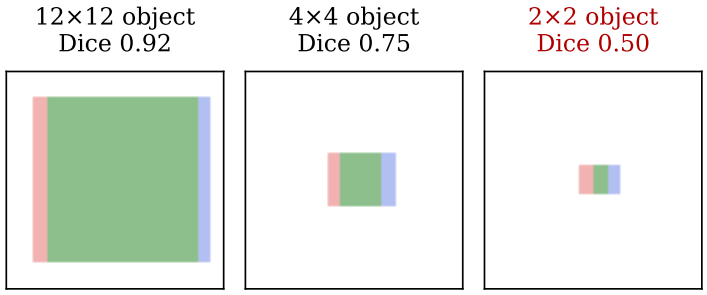
Left: fingerprint → rules, plus fixed defaults and a few cross-validated choices. Right: GPU memory is an input to the rules, so a different GPU gives a different patch and possibly a different winner.

The exam trap follows directly: **a different GPU can give a different "optimal" configuration**, because GPU memory feeds the patch-size rule (a different patch means a different network depth) and because non-deterministic training can flip the close empirical choice. **Loss Odyssey** (Ma 2021) asks which loss you should actually use. It sorts 20 segmentation losses into 4 families. **Distribution-based** losses compare per-voxel probabilities: CE, weighted CE, **TopK** (CE averaged over only the hardest k% of voxels, default 10%) and **focal** loss (CE multiplied by $(1-s)^{\gamma}$ so easy voxels count less; with γ=2, a voxel already at s=0.9 gets weight 0.01). **Region-based** losses compare overlaps: Dice, IoU, and **Tversky**, $\frac{TP}{TP+\alpha FP+\beta FN}$, which weighs false positives against false negatives (α=β=0.5 gives exactly Dice). **Boundary-based** losses compare contours (boundary loss, HD loss), and **compound** losses are weighted sums (Dice + CE, Dice + TopK). Benchmarked on four 3D CT tasks of increasing imbalance (liver, pancreas, liver tumour, multi-organ), **no single loss wins everywhere**, but **Dice-based compound losses are the most robust** first choice, and boundary losses work best paired with a region loss.



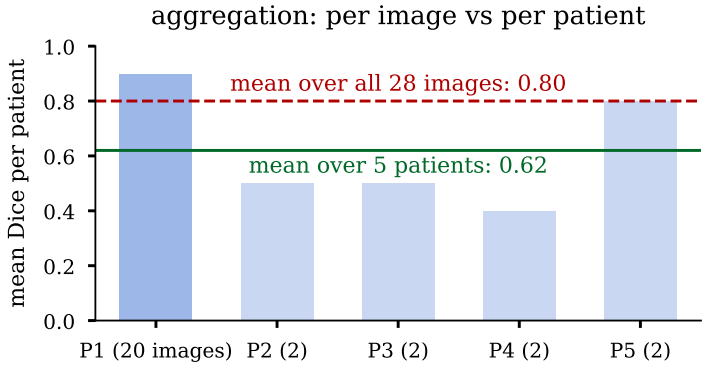
Left: the 4 loss families. Right: background-to-foreground voxel ratios of the benchmark tasks (from the paper); in the worst multi-organ class only 1 voxel in 1243 is foreground.

Measuring results is the second half of the story. **Metrics Reloaded** (Maier-Hein 2024) is a consensus framework, agreed through a multi-round expert vote (a **Delphi process**), for choosing metrics that match what you care about clinically. You first describe the task as a **problem fingerprint**: the clinical goal, the target structures (small? many? touching?), the data and the output type. The framework then guides you to suitable metrics for one of **4 problem categories**: **image-level classification** (one label per image), **object detection** (find and count objects), **semantic segmentation** (a class per pixel) and **instance segmentation** (a class per pixel *and* which object it belongs to). It names 3 kinds of pitfalls. The first is the **wrong problem category** (counting lesions is detection, so scoring it as segmentation hides missed lesions). The second is **poor metric selection**, e.g. Dice on tiny structures, where the same 1-pixel error that barely matters for a big organ destroys the score of a small lesion.



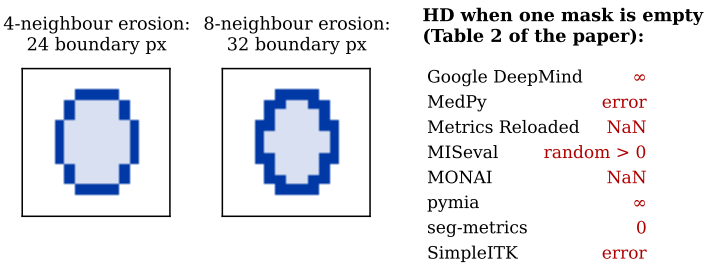
Same mistake (prediction shifted by 1 pixel), three object sizes: green = correct, red = missed, blue = extra. Dice drops from 0.92 to 0.50 just because the object is smaller.

The third is **poor metric application**, such as aggregating wrongly over a hierarchical dataset: if one patient contributes 20 images and four others contribute 2 each, the plain mean over images is dominated by that one patient.



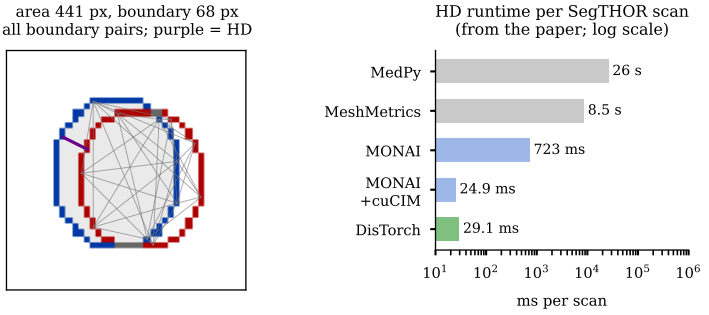
Averaging over all images gives 0.80, but the typical patient is much worse (0.62); define what one sample is before averaging.

Even the right metric can be computed wrongly. **Podobnik & Vrtovec (2025)** ran 11 open-source tools on the same masks and found differences in HD of more than 100 mm. So **a "statistically significant improvement" can come from just switching the metric tool**. They identify 3 causes. **Boundary extraction** differs: eroding with 4- or 8-neighbours, or shifting the grid by half a voxel, gives different boundary pixels and therefore different distances. **Mathematical definitions** differ (how the 95th percentile is taken, how distances are averaged), and one tool (MISeval) ignores the voxel size altogether. **Edge cases** differ: with an empty mask, tools return ∞, NaN, 0, an error, or even a random number.



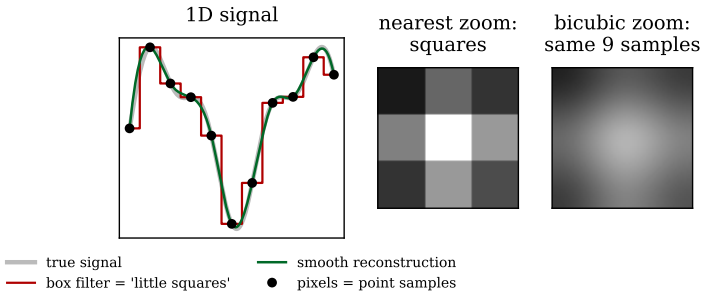
Left: two common boundary definitions give 24 vs 32 boundary pixels for the same mask. Right: what different tools return for HD when one mask is empty.

Their advice: follow Metrics Reloaded, **report the tool's name and version**, handle empty masks yourself, check units and voxel spacing, and on image grids prefer the Google DeepMind implementation, which was the most accurate. Distance metrics are also often skipped simply because they are slow, complex and pull in awkward dependencies. **DisTorch** (Rony & Kervadec 2025) fixes the speed with a brute-force idea. Instead of computing two Euclidean distance transforms, it measures every pairwise distance between the two boundaries directly, on the GPU, skipping voxels shared by both boundaries (their distance is 0). Quadratic cost sounds terrible, but a boundary is tiny compared to the volume (a 2D blob of 441 pixels has only 68 boundary pixels). The KeOps library (or Triton, according to the poster) also takes the min/max without ever storing the full distance matrix. The result is as fast as MONAI with cuCIM and orders of magnitude faster than CPU tools, in under 50 lines of code. It is slow only for random-looking predictions with huge boundaries, where HD is meaningless anyway. It is the tool used in assignment 3.



Left: HD is the longest of the shortest boundary-to-boundary distances (purple), found by checking boundary pairs. Right: runtime per SegTHOR scan (from the paper); DisTorch is about 1000× faster than MedPy.

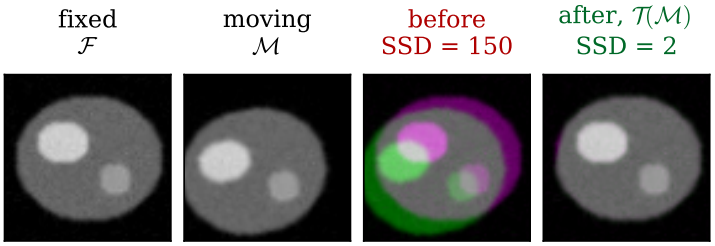
Finally, **Smith (1995), "A pixel is not a little square"**, explains why all these mm-level numbers are subtle. **A pixel is a point sample**: a measurement *at* a point, not a tiny tile of colour (and a voxel is not a little cube). To get a continuous image back from the samples you need a **reconstruction filter**. The box filter, which produces "little squares", is the crudest choice; smoother filters (linear, cubic) give a better approximation of the real signal. When you zoom into an image and see squares, you are only seeing each sample copied many times, not the true shape of a pixel. This matters for resampling, for distances between boundaries (which point of the "square" do you measure from?), and for any claim of sub-millimetre precision.



Left: the black dots are the pixels; the red staircase is the "little square" (box) reconstruction, the green curve a smooth one, close to the true grey signal. Right: the same 9 samples zoomed with nearest-neighbour (squares) and bicubic (smooth) interpolation.

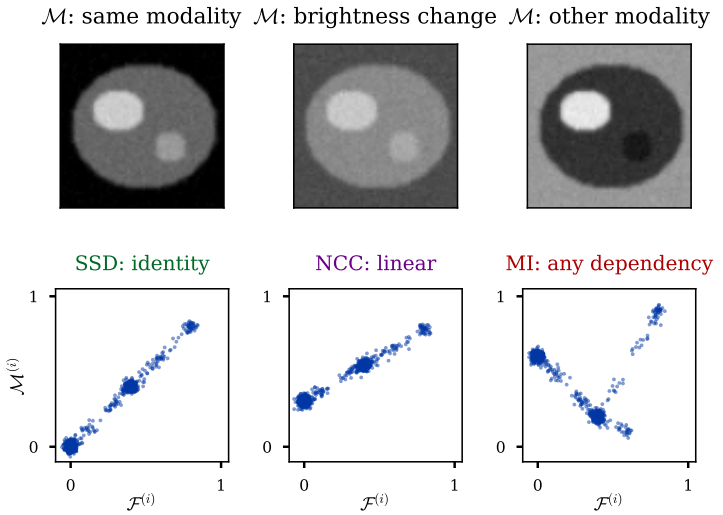
L3 Registration

Registration means aligning two images so that the same anatomy sits at the same coordinates. The **fixed** image $\mathcal{F}$ stays put, and the **moving** image $\mathcal{M}$ is transformed until it overlaps $\mathcal{F}$ (this week's CT onto last month's CT to see if a tumour grew; an MRI onto a CT to combine soft tissue and bone). Formally we look for the transform $\mathcal{T}$ that makes the two images most similar: $\min_{\mathcal{T}} \mathcal{L}(\mathcal{F}, \mathcal{T}(\mathcal{M}))$, where $\mathcal{T} : \Omega_2 \to \Omega_1$ maps positions of the moving image into the space of the fixed one. So, as in L2, it is an optimisation problem; only now we optimise the transform instead of network weights, and the loss $\mathcal{L}$ measures "how well aligned".

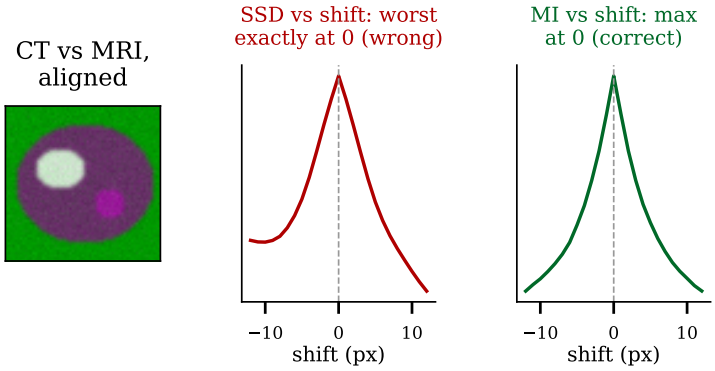


Overlay: fixed in magenta, moving in green; where they agree the colours mix to grey. After finding $\mathcal{T}$ the colours disappear and SSD drops from 150 to 2.

The choice of $\mathcal{L}$ depends on how the two images' intensities relate. **SSD** (sum of squared differences) $\sum_i (\mathcal{F}^{(i)} - \mathcal{T}(\mathcal{M})^{(i)})^2$ assumes the **same tissue has the same brightness** in both, so it suits the **same modality** (CT–CT); **SSD is the only one not robust to intensity changes**. **NCC** (normalised cross-correlation) only assumes a **linear** relation ($\mathcal{M} = a\mathcal{F} + b$), so it survives global brightness or contrast changes. **NGF** (normalised gradient fields) compares the *directions* of edges, so it works across modalities as long as the edges coincide. **MI** (mutual information) only assumes that **knowing one intensity tells you something about the other** ("CT bright" always goes with "MRI dark"), whatever the relation: $\mathrm{MI} = H(\mathcal{F}) + H(\mathcal{T}(\mathcal{M})) - H(\mathcal{F}, \mathcal{T}(\mathcal{M}))$. Entropy H measures unpredictability; when the images are aligned the joint histogram is sharp, the joint entropy $H(\mathcal{F}, \mathcal{T}(\mathcal{M}))$ is low and MI is high, so MI is the standard choice for **multi-modal** pairs (CT–MRI, PET–MRI). **MIND** describes each voxel by how similar it is to its neighbours (a self-similarity pattern), which stays the same even across very different modalities.

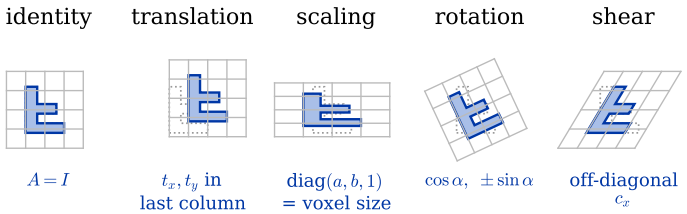


Each dot is one voxel of the aligned pair: its intensity in $\mathcal{F}$ against its intensity in $\mathcal{M}$. Same modality: a diagonal line (SSD). Brightness change: a straight line with another slope (NCC). Other modality: a curved but still predictable relation (MI).



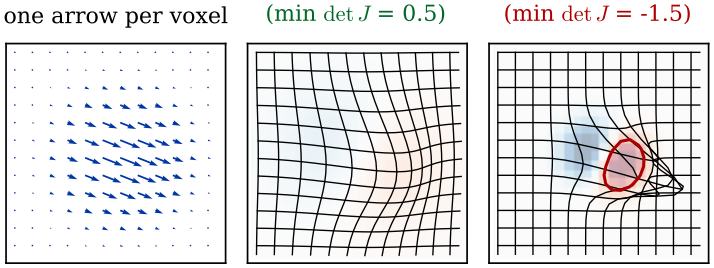
Sliding the MRI over the CT: SSD is worst *exactly at the correct alignment, because bright CT tissue is dark in MRI. MI peaks at the correct alignment.*

Before transforming anything, note that a scan file already contains a transform. Its metadata (NIfTI/DICOM) stores a 4×4 **affine matrix** that maps **voxel coordinates** (row, column, slice) to **world/patient coordinates** in mm: $(x', y', z', 1)^\top = A\,(x, y, z, 1)^\top$. The extra "1" (**homogeneous coordinates**) is a trick that lets a translation be a matrix multiplication too. The matrix encodes **voxel size** (scaling), **rotation**, **position** in the scanner (translation) and shear, **not whether a registration is rigid or elastic**. Each building block has a simple shape: translation = identity with (t_x, t_y, t_z) in the last column; scaling = $\mathrm{diag}(a, b, c, 1)$ (voxel size $1{\times}1{\times}2.5\,\mathrm{mm} \rightarrow \mathrm{diag}(1, 1, 2.5, 1)$); rotation = a block of $\cos\alpha$ and $\pm\sin\alpha$; shear = off-diagonal terms. They combine by multiplication, $A = C \cdot R \cdot S \cdot T$.



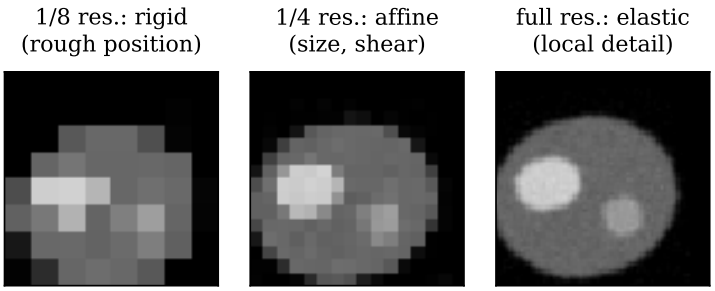
The four building blocks applied to a grid and a shape (dotted = before). An affine transform keeps straight lines straight and parallel lines parallel.

The simplest registration searches over these matrices: $\min_A \mathcal{L}(\mathcal{F}, \mathcal{T}_A(\mathcal{M}))$. Strictly, **rigid** means only rotation + translation (the object keeps its shape: 6 numbers in 3D), while **affine** also allows scaling and shear (12 numbers). Rigid is right for things that do not deform: **adult bones** (skull, femur, a single rib), or a patient who simply moved (**at least rigid**). Most of the body is soft, though: **lungs** change with breathing, the **heart** beats, **kidneys** shift with breathing, and the **ribcage as a whole** moves even if each rib is rigid. And **teenagers scanned years apart have grown bones, so even bones need affine or non-rigid**. **Elastic** (non-rigid) registration therefore lets every voxel move on its own: a **deformation field** gives one displacement vector per voxel, $\mathcal{T} : \Omega_2 \rightarrow \mathbb{R}^3$. With millions of free vectors the problem is **ill-posed**: many crazy fields match the intensities equally well (voxels could swap places freely). We add a **regulariser**, $\min_{\mathcal{T}} \mathcal{L}(\mathcal{F}, \mathcal{T}(\mathcal{M})) + \lambda\mathcal{R}(\mathcal{T})$, that asks for **smoothness** (neighbouring voxels move alike, like a rubber sheet rather than sand) and **no folds**. Folds are measured with the **Jacobian determinant** $\det J$, the local volume change: 1 = unchanged, 2 = doubled, 0.5 = halved, and $\det J \leq 0$ **= a fold**, space flipping over itself, which is physically impossible. So we want $\det J > 0$**, ideally close to 1**.



Left: a deformation field (one arrow per voxel). Middle: a gentle field warps the grid smoothly ($\det J > 0$ everywhere). Right: a too-strong field makes grid lines cross; inside the red contour $\det J \leq 0$, a fold.

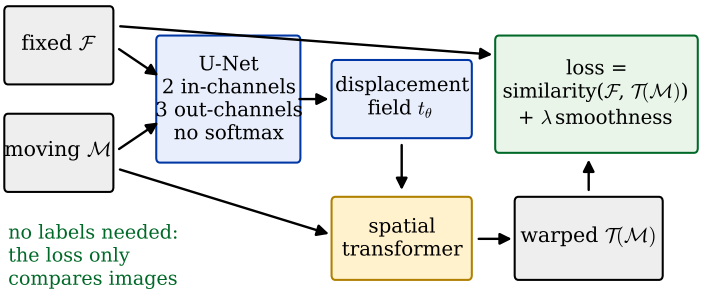
Because the problem is so hard, it is broken into easier steps. Toolboxes such as **elastix** chain **rigid** → **affine** → **elastic**, each starting from the previous result, and use a **pyramidal** (coarse-to-fine) approach: first align heavily downsampled images, where only the big picture is visible and the optimiser cannot get lost in details, then refine at higher resolution.



Pyramid: solve the easy coarse problem first, then use it as the starting point at the next resolution.

Classical registration runs a slow optimiser for *every* new pair of images. **Learned registration** trains a network once and then predicts a deformation in one fast forward pass. **VoxelMorph** is a U-Net that takes $(\mathcal{F}, \mathcal{M})$ and outputs a deformation field. To turn the **segmentation U-Net into a registration U-Net**: use **2 input channels** ($\mathcal{F}$ and $\mathcal{M}$), **3 output channels** (the x, y, z displacement per voxel), **no softmax** (displacements are any real numbers in mm, not probabilities), and add a **spatial transformer** that warps $\mathcal{M}$ with the predicted field in a differentiable way, so gradients can flow back. The loss is the same as in classical registration, $\mathrm{similarity}(\mathcal{F}, \mathcal{T}(\mathcal{M})) + \lambda\,\mathrm{smoothness}$, now summed over a dataset of pairs. That means **no labels are needed**: training is unsupervised, because the images supervise themselves. Segmentations or landmarks can be added as extra supervision (the warped liver mask should overlap the fixed liver mask), and a no-fold term can be added to the loss. The trade-off: **classical** methods give the best, verifiable result for each pair but are slow; **learned** methods are fast at inference but come with **no guarantee of correctness** on a new pair.

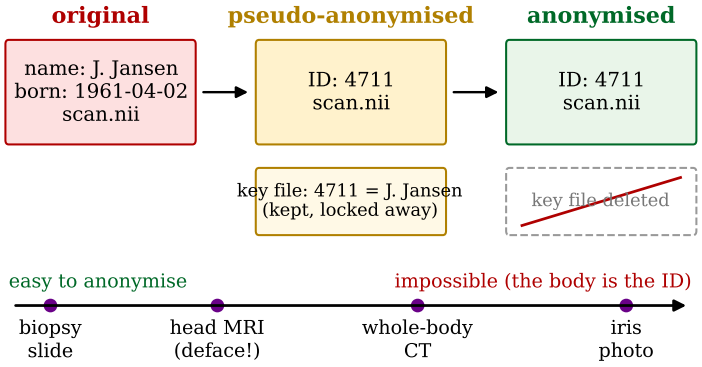
The Learn2Reg challenge benchmarks both.



VoxelMorph: a U-Net predicts the displacement field, a spatial transformer warps $\mathcal{M}$, and the loss compares the warped image with $\mathcal{F}$ (+ smoothness). No ground-truth deformation is needed.

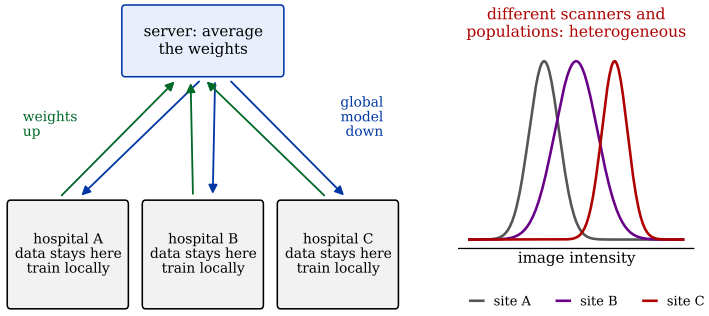
L6 Ethics, data, reproducibility

Medical images belong to people, so working with them is regulated. In Europe the **GDPR** requires **consent** from the patient, a clear **purpose** for which the data is used, and **careful handling**; these rules are still being debated and changed, but they cannot be ignored, especially in industry. A common trap concerns **retrospective studies**, which reuse scans that were made for patient care (10 years of liver CTs in the hospital archive): they **need consent for this new use**. Exceptions exist when consent **cannot be asked** (the patient has died) or **cannot reasonably be required** (thousands of former patients who cannot be traced), but **the ethics committee decides, not you** (in the Netherlands the METC, under the WMO law). Before collecting anything you write a **data management plan (DMP)** that says how the data will be kept secure, how its quality is guaranteed and how you comply with the law. Then separate **data** (the image itself) from **metadata** (name, date, sex, weight). Replacing the name by a random ID makes the data **pseudo-anonymised** if the table linking IDs to names still exists somewhere, and **anonymised** only once that table has been deleted.



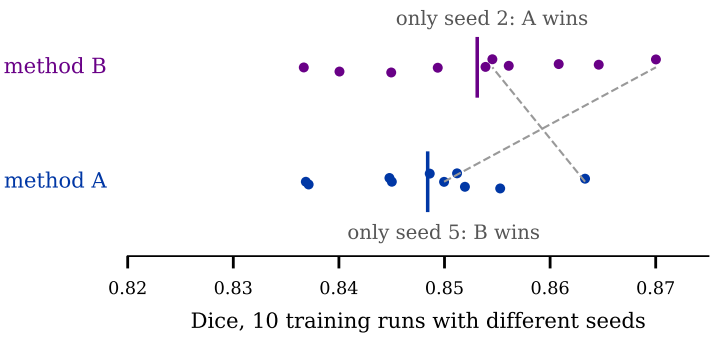
Top: pseudo-anonymised data can still be linked back through the key file; anonymised data cannot. Bottom: how hard anonymisation is depends on how much of the body the image shows.

Even anonymised data has limits, because of **re-identification**: someone can match the data with other information (a 3D rendering of a head MRI shows the face, which can be matched to a photo; a rare implant plus age plus hospital may single out one person). Bodies are unique; in the lecture even finger lengths worked as a fingerprint. Mitigations are **skull stripping/defacing** (removing the face from head scans) and **cropping** to the region of interest, but anonymisation is **impossible for the iris, the whole body or the body surface**, where the image *is* the identity. Hence the exam ordering **biopsy scan < head MRI < whole-body CT < iris photo**, from easiest to hardest. Also remember that **children are not small adults**: their proportions and anatomy differ, so models and rules made for adults do not transfer. When sharing, read the **Creative Commons licence**: **BY** = give attribution, **NC** = non-commercial only, **SA** = share derived work under the same licence, **ND** = no modified versions (a CC-BY-NC dataset cannot be used to train a model you sell). Between institutions you usually need a **data transfer agreement (DTA)**, and not every project member needs access to the data. The **safest data is the data you don't have**: collect only what you need, and if a breach happens your institution must report it as soon as possible. Basic hygiene: **no patient data on GitHub**, passphrases plus a password manager, two-factor authentication, encrypted drives, and never plug in a USB stick you found. Sometimes data simply cannot leave the hospital. **Federated learning** then trains a model anyway: each site trains locally and sends only *weights*, never images, to a server that averages them and sends the global model back. **Differential privacy** adds carefully chosen noise during training so that the final model does not reveal whether any single patient was in the data. Neither is free: both mean **lower performance and higher compute**, and **federated learning underperforms when sites are heterogeneous** (different scanners, protocols and populations), because the local updates pull the shared model in different directions.



Left: federated learning moves weights, not data. Right: when each hospital's images look different, averaging their updates gives a compromise model that fits none of them perfectly.

Reproducibility can mean four increasingly loose things: bit-identical weights; the same code and data retrained on another machine; the same with other package versions; or results within a stated confidence interval. Training is full of randomness: weight **initialisation**, the order of **batch sampling**, **non-deterministic GPU operations** (floating-point sums done in a different order give slightly different results), and even hardware and kernel versions. Computers use **pseudo-random** numbers generated from a **seed**, so you can remove much of this by fixing the seeds (`random.seed(0)`, `np.random.seed(0)`, `torch.manual_seed(0)`) and enabling `torch.use_deterministic_algorithms`, plus pinned package versions and shared code and model. Above all, **train several runs**, otherwise you may be comparing seeds rather than methods. Note that **public code + private data = only partially reproducible**, since nobody else can rerun it. **Public challenges** fix this for a field: everyone gets the same data, the same metrics and a hidden test set, which makes comparison fair and the data openly available.

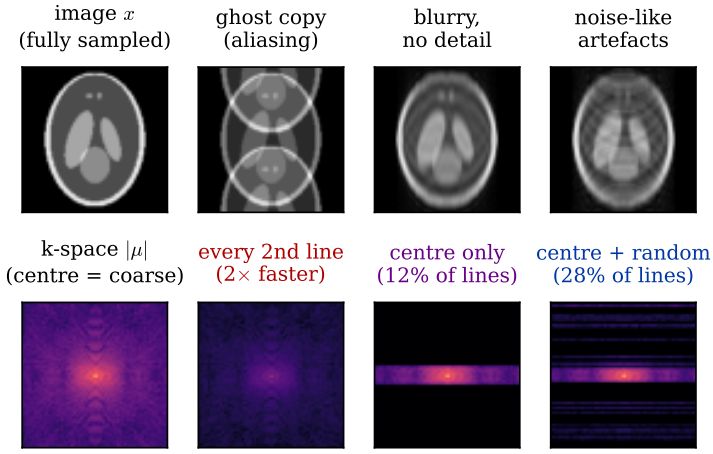


Two methods, 10 seeds each. Picking a single seed can make either one "win"; only the spread over runs shows that the difference is within noise.

Finally, **deployment and responsibility**. Software has long escaped the responsibility culture of other engineering fields ("move fast and break things"), but in medicine bugs hurt people (Therac-25, a radiotherapy machine whose software bug caused lethal overdoses). In 2016 Hinton said we should stop training radiologists because AI would beat them within five years; **that prediction was wrong**. The current consensus is that **radiologists with AI do better than radiologists without**, and that AI will not replace them but can speed up tasks, reduce the need for a second opinion and make grading more consistent. **Who is responsible** when an AI-assisted decision goes wrong is still unclear: the data collectors, the engineers, the company, the regulator that approved it, or the user? Aviation offers lessons: **automation can erode intuition** (Air France 447: when the autopilot switched off, the crew could no longer fly the plane manually), and **confirmation bias** makes people trust what they expect to see (Air Transat 236: the crew blamed a faulty sensor while the plane was leaking fuel). Add **fairness**, **robustness** and **interpretability** to the list of things a deployed model must be checked for.

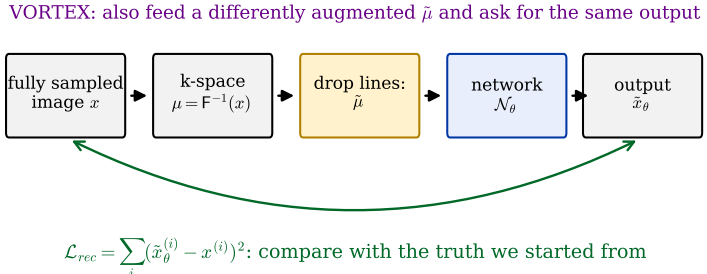
L8 Reconstruction

Recall from L1 that MRI does not measure pixels: it measures frequencies, stored in **k-space**, and the image is reconstructed from them with a Fourier transform. Measuring every line of k-space takes long (20–60 minutes), so the obvious speed-up is to **undersample**: measure only some lines (every second line → 2× faster). The price is **artefacts**, and what they look like depends on which lines you skip. Skipping every other line folds the image onto itself (**aliasing**, a ghost copy); keeping only the centre keeps the coarse picture but loses all detail (the centre of k-space holds overall shapes and contrast, the outside holds edges, the same idea JPEG uses to compress photos); keeping the centre plus random lines spreads the error out as noise-like artefacts, which is the easiest kind to clean up.



Bottom: which k-space lines are measured. Top: the resulting image after a plain inverse Fourier transform. Every pattern saves time but costs something different.

Recovering a clean image from undersampled k-space is **ill-posed**: many images agree with the lines you measured. The classical answer is **compressed sensing**, an **iterative** algorithm that searches for the image that fits the measurements *and* is "simple" (e.g. sparse in some representation). The learned answer, **learned reconstruction**, trains a network to do it. Notation: Ω is image space, Ξ the spectral domain (k-space), $\mathsf{F} : \Xi \to \Omega$ the Fourier transform from k-space to image, so the k-space of an image is $\mu_n = \mathsf{F}^{-1}(x_n)$. The training data costs nothing extra: take **fully sampled** scans, compute their k-space, **simulate the undersampling** by deleting lines, and train the network $\mathcal{N}_\theta$ to map that undersampled k-space $\tilde{\mu}$ back to the original image, with $\mathcal{L}_{rec} = \sum_i (\tilde{x}_\theta^{(i)} - x^{(i)})^2$, i.e. SSD between the output and the truth you started from. **VORTEX** adds self-supervision: give the network two differently augmented versions of the same undersampled data (different noise, a small motion) and add a **consistency loss** asking for the same image from both, $\min_\theta \mathcal{L}_{rec} + \lambda \mathcal{L}_{cons}$, which makes it robust to such variations.

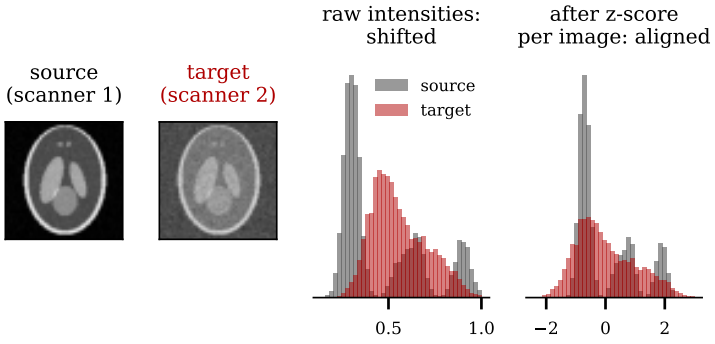


Learned reconstruction: the ground truth is the fully sampled image itself, so supervision comes for free.

The same recipe, "degrade a good image, learn to undo it", powers **denoising**, **super-resolution**, **modality translation** (predict a CT-like image from an MRI) and **anomaly detection** (a model that only learned to rebuild healthy anatomy fails to rebuild a tumour, and that failure shows where it is). All of them share one hard **limit: a network cannot recover information that was never measured**. It can only fill gaps with what is *typical*, which may look convincing yet be wrong for this patient; and different modalities exist precisely because they measure different things.

L8 Foundation models & domain shift

A **foundation model** is a general model trained on a very large dataset and meant to be reused for many tasks. The word is used loosely, so ask four questions: is it **promptable**? trained on **medical** or natural images? **fine-tuned** for a specific use? trained **supervised** or **self-supervised**? The best-known one is **SAM** (Segment Anything): you give it a **prompt** (a click, a box, or text) and it segments the object there. It was trained on **2D natural images**; **SAM2** adds a video mode, which can be used to treat a **3D scan as a video** of slices, and **SAM3** also includes some public medical data. **MedSAM** is SAM **fine-tuned** on medical images. The exam point is the trade-off: **fine-tuning raises in-domain performance but lowers generalisation**, so on a **private CT dataset from another hospital, plain SAM beat MedSAM**; MedSAM had become a specialist for the data it was tuned on. **nnInteractive** (from the nnU-Net team) is natively 3D and **more stable under prompt changes**: two annotators clicking a few pixels apart get nearly the same mask, which is not true for SAM. Other reusable resources: **TotalSegmentator**, both a trained model (Apache licence) and a large dataset (CC-BY) of CTs and MRIs, and **Virchow**, a histopathology model trained on 1.5 million slides and used to turn image patches into **embeddings** (feature vectors) for downstream tasks. All of this runs into **domain shift**: a model trained on data from domain 1 is used on data from domain 2, and performance drops. Typical sources are a different **scanner**, **vendor** or **protocol**, a different **operator**, or a different **population** (children, another country, another disease stage). **Ultrasound generalises worse than CT** because the image depends on the operator's hand.



The same anatomy on two scanners: raw intensity distributions differ (a model trained on the grey one sees unfamiliar values). Normalising each image (subtract its mean, divide by its standard deviation) brings them much closer.

Domain adaptation adapts a model from a **source** domain (labelled) to a **target** domain (new, often unlabelled); in **source-free** adaptation the source data itself is no longer available, only the trained model. Practical fixes: **transfer learning** (start from a pretrained model and fine-tune it on a little target data), domain adaptation, **intensity normalisation**, **augmentation** that mimics other scanners (contrast, noise, resolution), and training on **multi-site** data from the start. Two exam scenarios: from **brain MRI to lung CT, fine-tune the existing model rather than retrain from scratch**, since low-level features (edges, textures) transfer; with **one big and two small hospitals**, use **federated learning** (L6), or share the big hospital's *weights* and **fine-tune locally** at each small one.